

Chapter 7

Sorting

7.1 Motivation

In this chapter, we use the term **list to mean a collection of records**, each record having one or more fields. The fields used to distinguish between among records are known as **keys**.

Examples of uses of sorting

(1) As an aid in searching. For example, **binary search** a sorted list with n records in $O(\log n)$, much better than **sequential search** a unsorted list in $O(n)$.

(2) As a means for matching entries in lists.

Comparing two lists of n and m records containing data that are essentially the **same** but from different sources: If we **first sort** the two lists, then we can do it in $O(t_{\text{sort}(n)} + t_{\text{sort}(m)} + n + m)$, otherwise we need $O(nm)$ time.

Actually there are much more applications of sorting, such as optimization, job scheduling, etc..

Consequently, sorting problem has been extensively studied, and there are many methods proposed. Unfortunately, **no one sorting method is the best for all applications.**

We shall therefore study several typical methods, indicating when one is superior to others.

Now let us **formally** state the sorting problem.

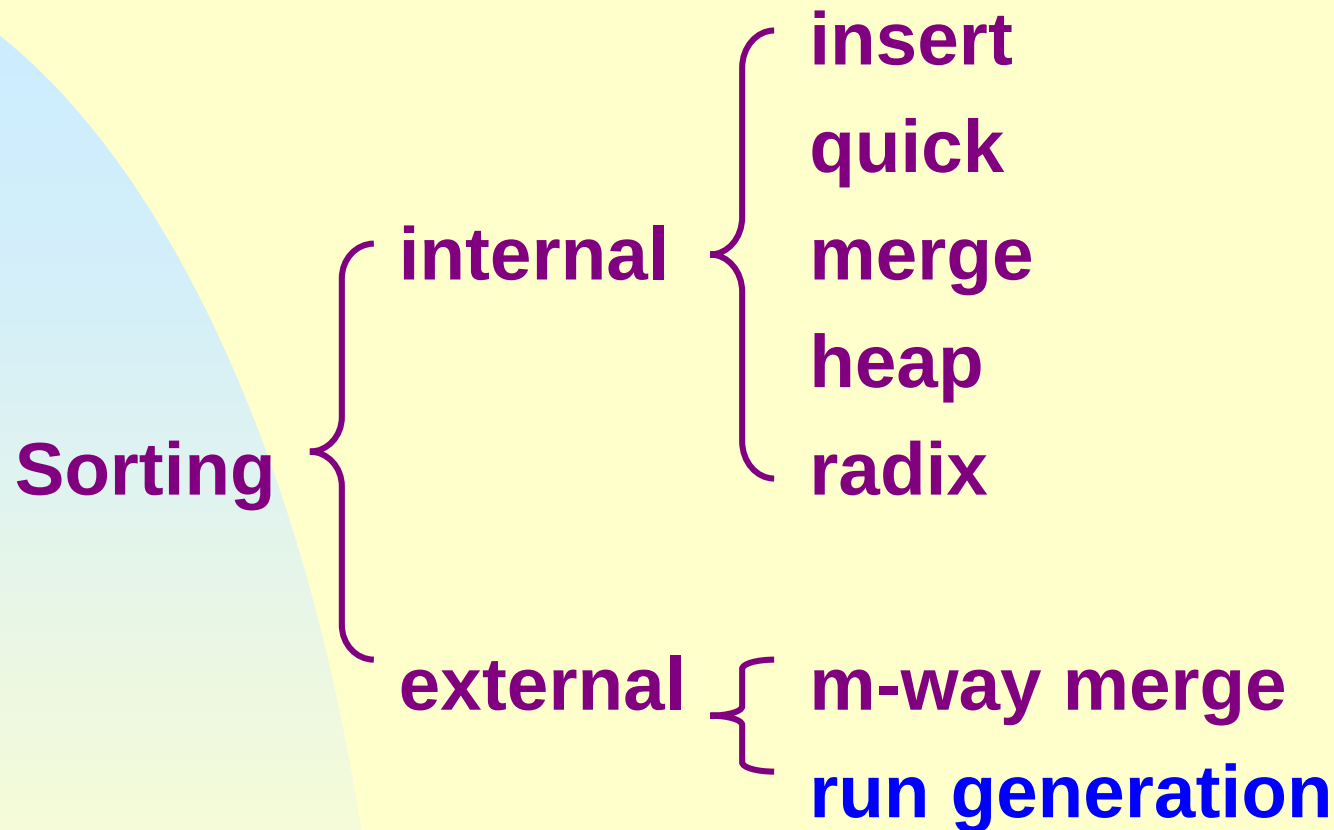
Formal statement for the sorting problem

- We are given a list of records $(R_1, R_2, \dots, R_n)$;
- Each record R_i has key value K_i ;
- There is a **totally ordered relation** ($\leq$) on the keys.
- A **totally ordered relation** (also called a linear order) on a set S is a binary relation $\leq$ (read as "less than or equal to") that satisfies the following four properties for all elements $a, b, c \in S$:
 - ✓ Reflexivity (自反性): $a \leq a$
 - ✓ Anti-symmetry (反对称性): If $a \leq b$ and $b \leq a$, then $a = b$
 - ✓ Transitivity (传递性): If $a \leq b$ and $b \leq c$, then $a \leq c$
 - ✓ Comparability (完全可比性): Either $a \leq b$ or $b \leq a$
- Then, the **sorting problem** is to find a **permutation** σ , such that $K_{\sigma(i)} \leq K_{\sigma(i+1)}$ for $1 \leq i \leq n-1$. The desired ordering is $(R_{\sigma(1)}, R_{\sigma(2)}, \dots, R_{\sigma(n)})$.

Stable sorting

- A sorting method is **stable**, if any $i < j$ and $K_i == K_j$ in the input list, then R_i precedes R_j in the sorted list.

The sorting methods that will be studied



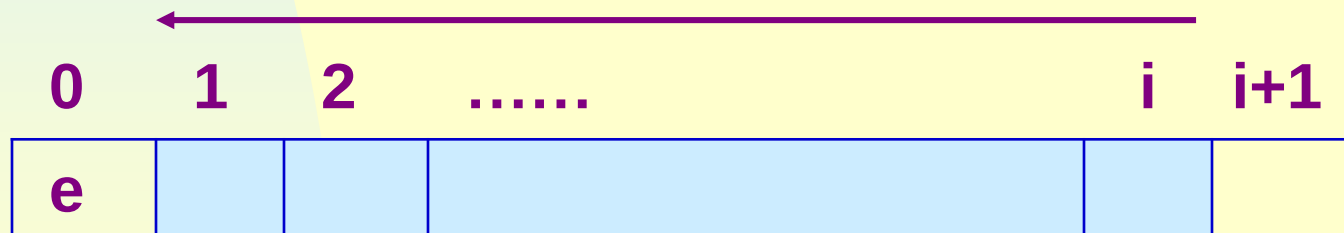
Hereafter, we assume that relational operators have been overloaded so that record comparison is done by comparing their keys.

7.2 Insertion Sort

Basic idea: In every step (pass), we **insert** a new record **e** into a **sorted sequence** of **i** records such that the resulting **sequence** of size **i+1** is also **ordered**.

Usually, the records are stored in an array **a**.

To simply realize a **insert step**, we use **a[0]** to keep the new record **e**.



Function **Insert** (Program 7.4) accomplishes this insertion.

Program 7.4: Insert a new element into a sorted list

```
template<class T>
```

```
void Insert(const T& e, T *a, int i)
```

```
{ //insert e into the ordered sequence a[1:i] such that the  
  // resulting sequence a[1:i+1] is also ordered. The array a  
  // must have space for at least i+2 elements.
```

```
  a[0]=e;
```

```
  while (e < a[i])  //stable
```

```
  {
```

```
    a[i+1]=a[i];
```

```
    i--;
```

```
  }
```

```
  a[i+1]=e; // i+1 is the insertion point.
```

```
}
```


Insert Sort Algorithm

In **Insertion Sort**, we begin with the ordered sequence **a[1]** and successively **insert** the records **a[2], a[3], ..., a[n]**.

Program 7.5: Insertion Sort

```
template<class T>
void InsertionSort (T *a, const int n)
{ //sort a[1:n] into nondecreasing order.
    for (int j=2; j<=n; j++) {
        T temp = a[j];
        Insert (temp, a, j-1);
    }
}
```

InsertionSort is stable.

Analysis of InsertionSort:

- The best case--- $O(n)$

- The worst case

Insert(e, a, i) makes $i+1$ comparisons before making insertion --- $O(i)$.

InsertionSort invokes Insert(e, a, i) for $i=j-1=1, 2, \dots, n-1$, so the overall time is

$$O\left(\sum_{i=1}^{n-1} (i+1)\right) = O(n^2).$$

- The average case

It can also be shown that the average time is $O(n^2)$.

The computing time based on relative disorder

- We can also estimate of the computing time of Insertion Sort based on relative disorder in the input list.
- Definition: R_i is left out of order (LOO) iff $R_i < \max_{1 \leq j < i} \{ R_j \}$.
- The insertion step has to be carried out only for those records that are LOO. If k is the number of LOO records, the computing time is $O(kn)$.

Example 7.1: $n=5$, key sequence is **5, 4, 3, 2, 1**

j	[1]	[2]	[3]	[4]	[5]
-	5	4	3	2	1
2	4	5	3	2	1
3	3	4	5	2	1
4	2	3	4	5	1
5	1	2	3	4	5

The worst case: every record are LOO.

Example 7.2: $n=5$, key sequence is 2, 3, 4, 5, **1**

j	[1]	[2]	[3]	[4]	[5]
-	2	3	4	5	1
2	2	3	4	5	1
3	2	3	4	5	1
4	2	3	4	5	1
5	1	2	3	4	5

The case that only R_5 is LOO.

The time for $j=2,3,4$ is $O(1)$

whereas for $j=5$ is $O(n)$.

Two important features of insertion sort

- When the number of L/Os $k \ll n$, it is very desirable.
- For small $n \leq 30$, it is the fastest.

Variations:

(1) Binary Insert Sort:

- ✓ Replacing the sequential searching technique used in Insert with **binary search** to reduce the number of comparisons.
- ✓ The **number of record moves** remains unchanged.

(2) Linked Insert Sort:

- ✓ The elements of the list are represented as a **linked list** rather than as an array.
- ✓ The **number of record moves becomes zero** because only the linked fields require adjustment.
- ✓ However, we must retain the **sequential search** used in Insert.

7.3 Quick Sort

The quick-sort has the **best average** behavior among the sorting methods we shall be studying.

Idea: given a list $(R_{\text{left}}, R_{\text{left}+1}, \dots, R_{\text{right}})$,

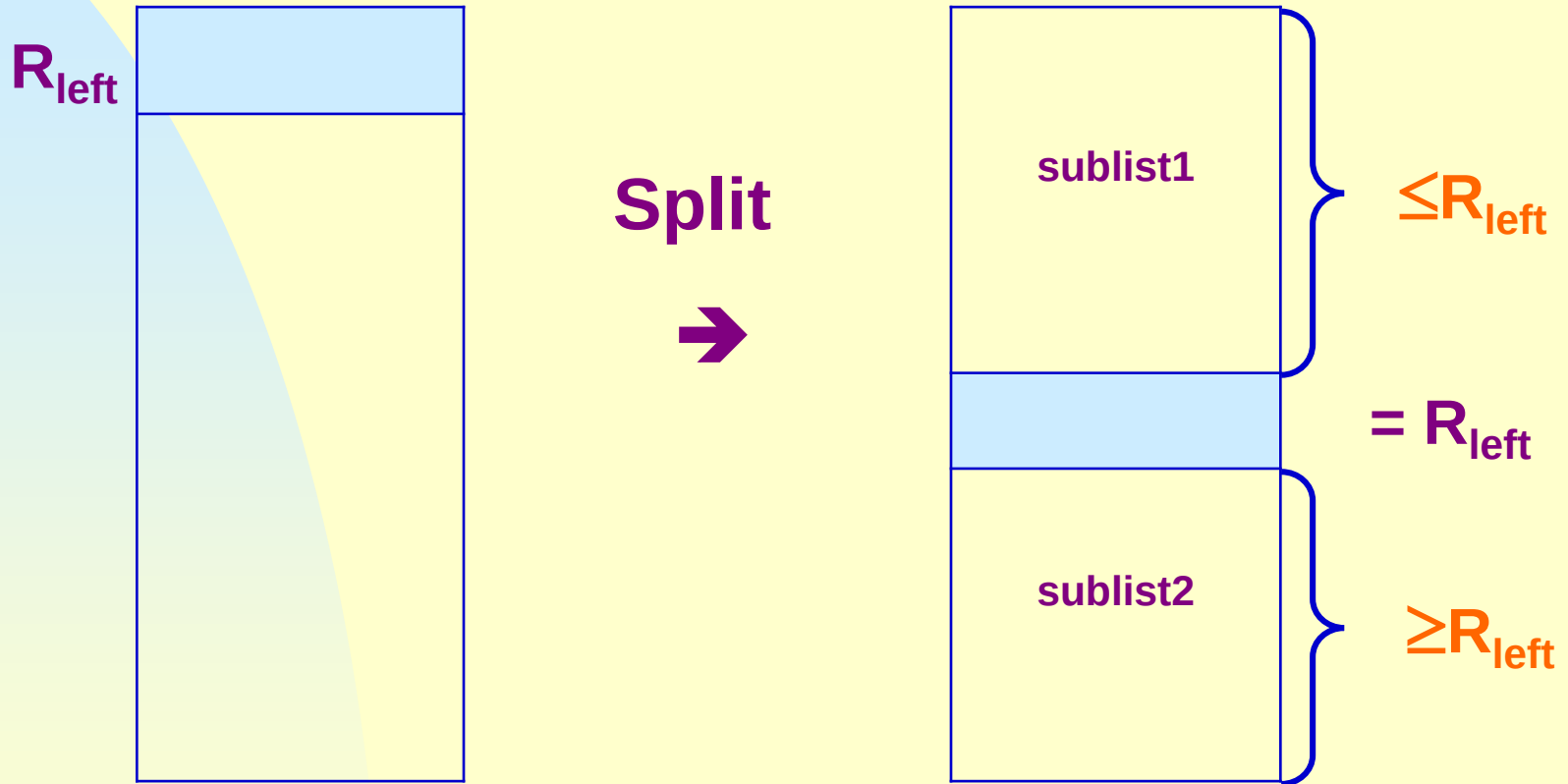
- ✓ select a **pivot** record from among R_i ($\text{left} \leq i \leq \text{right}$), put the pivot in the correct spot $\sigma(\mathbf{s(\text{pivot})})$ such that after positioning,

$$R_{\sigma(j)} \leq R_{\sigma(\mathbf{s(\text{pivot})})} \quad \text{for } \sigma(j) < \sigma(\mathbf{s(\text{pivot})})$$

$$R_{\sigma(\mathbf{s(\text{pivot})})} \leq R_{\sigma(j)} \quad \text{for } \sigma(\mathbf{s(\text{pivot})}) < \sigma(j)$$

- ✓ finally, the records in the left of the pivot and those in the right are sorted independently.

For example, we choose $R[\text{left}]$ as the pivot. Then we get two sublist1 and sublist2:



Finally sublist1 and sublist2 are sorted independently.

Program 7.6: Quick Sort

```
template <class T>
```

```
void QuickSort (T *a, const int left, const int right)
{ // Sort a[left:right] into non-decreasing order. a[left] is
  // chosen as the pivot. Assume we have a[right+1] and
  // a[left] ≤ a[right+1].
```

```
  if (left < right) {
```

```
    int i=left, j=right+1, pivot=a[left];
```

```
    do {
```

```
      do i++; while (a[i]<pivot); //while will terminate
```

```
      do j--; while (a[j]>pivot); //while will terminate
```

```
      if (i<j) swap(a[i], a[j]); //may be i<j or i=j or i=j+1
```

```
    } while (i<j); //after while a[i] ≥ pivot ≥ a[j] and i=j or i=j+1
```

```
    swap(a[left], a[j]); //swap(a[left], a[i]) is wrong
```

```
    QuickSort(a, left, j-1);
```

```
    QuickSort(a, j+1, right);
```

```
  } //end of if
```

```
}
```



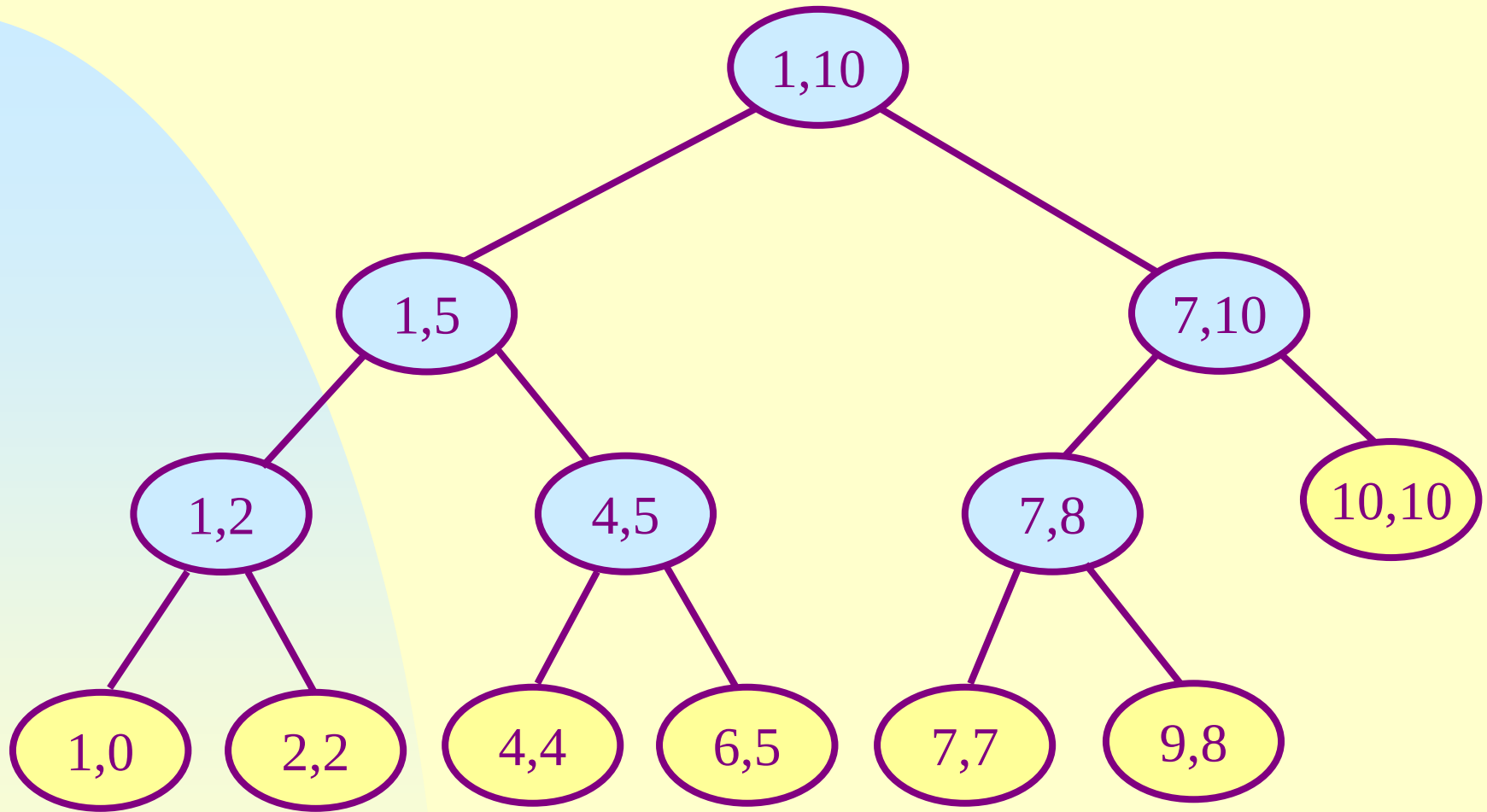
To sort $a[1:n]$, invoke $\text{QuickSort}(a, 1, n)$.

Example 7.3:

R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}	left	right
[26	5	37	1	61	11	59	15	48	19]	1	10
[11	5	19	1	15]	26	[59	61	48	37]	1	5
[1	5]	11	[19	15]	26	[59	61	48	37]	1	2
1	5	11	[19	15]	26	[59	61	48	37]	4	5
1	5	11	15	19	26	[59	61	48	37]	7	10
1	5	11	15	19	26	[48	37]	59	[61]	7	8
1	5	11	15	19	26	37	48	59	[61]	10	10
1	5	11	15	19	26	37	48	59	61		

The element with the orange color is the **pivot**.

Recursive call tree of quicksort (in preorder)



The left number and the right number in each node is the parameter left and parameter right of quicksort, respectively.

Analysis of QuickSort:

(1) The worst case

2 sublists with size $n-1$ and 0 .

$$T_{\text{worst}}(n) \leq cn + T_{\text{worst}}(n-1)$$

$$\leq cn + c(n-1) + T_{\text{worst}}(n-2)$$

...

$$\leq c \sum_{i=1}^n i + T_{\text{worst}}(0) = cn(n+1)/2 + d = O(n^2).$$

(2) The optimal case

2 sublists with **equal size**, roughly $n/2$.

$$T_{\text{opt}}(n) \leq cn + 2T_{\text{opt}}(n/2)$$

$$\leq cn + 2(cn/2 + 2T_{\text{opt}}(n/4))$$

$$\leq 2cn + 4T_{\text{opt}}(n/4)$$

...

$$\leq cn \log_2 n + nT_{\text{opt}}(1) = O(n \log n).$$

(3) The average time

Lemma 7.1: $T_{\text{avg}}(n) \leq k \cdot n \log_e n$ for $n \geq 2$.

Proof:

In the call to QuickSort(a, 1, n), the pivot gets place at position j, we need to sort 2 sublists of size j-1 and n-j. j may take on any of the 1 to n **with equal probability**, so

$$\begin{aligned} T_{\text{avg}}(n) &\leq cn + \frac{1}{n} \sum_{j=1}^n (T_{\text{avg}}(j-1) + T_{\text{avg}}(n-j)) \\ &= cn + \frac{2}{n} \sum_{j=0}^{n-1} T_{\text{avg}}(j), n \geq 2 \text{ --- (7.1)} \end{aligned}$$

Let $T_{\text{avg}}(0) \leq b$ and $T_{\text{avg}}(1) \leq b$ for some constant b ,
and $k=2(b+c)$,
we show $T_{\text{avg}}(n) \leq kn \log_e n$ for $n \geq 2$.

Induction on n .

$n=2$, from (7.1) $T_{\text{avg}}(2) \leq 2c+2b \leq k*2 \log_e 2$.

Assume $T_{\text{avg}}(n) \leq k*n \log_e n$ for $2 \leq n < m$.

Then

$$T_{avg}(m) \leq cm + \frac{2}{m} \sum_{j=0}^{m-1} T_{avg}(j)$$

$$\leq cm + \frac{4b}{m} + \frac{2}{m} \sum_{j=2}^{m-1} T_{avg}(j)$$

$$\leq cm + \frac{4b}{m} + \frac{2k}{m} \sum_{j=2}^{m-1} j \log_e j \text{ --- (7.2)}$$

Since $j \log_e j$ is an increasing function of j , Eq. (7.2) yields

$$\begin{aligned} T_{\text{avg}}(m) &\leq cm + \frac{4b}{m} + \frac{2k}{m} \int_2^m x \log_e x dx \\ &= cm + \frac{4b}{m} + \frac{2k}{m} \left[\frac{m^2 \log_e m}{2} - \frac{m^2}{4} \right] \\ &= cm + \frac{4b}{m} + km \log_e m - \frac{km}{2} \\ &= km \log_e m + \left[cm + \frac{4b}{m} - \frac{km}{2} \right] \end{aligned}$$

For $m \geq 2$

$$\begin{aligned} cm + \frac{4b}{m} - \frac{km}{2} &= cm + \frac{4b}{m} - \frac{2(b+c)m}{2} \\ &= \frac{4b}{m} - bm \leq 0 \end{aligned}$$

Therefore For $m \geq 2$, $T_{\text{avg}}(m) \leq km \log_e m$.

Stack space requirement:

- ✓ For the case that is the best for time complexity:
 - **split evenly**--- $O(\log n)$
- ✓ For the case that is the worst for time complexity:
 - split into a **left sublist of size $n-1$** and a right of 0 --- $O(n)$
 - split into a **left sublist of size 0** and a right of $n-1$ --- $O(1)$
- ✓ If we sort the **smaller sublist first**, then
 - for the case that is best for time complexity--- **$O(\log n)$**
 - for the case that is worst for time complexity--- $O(1)$

Variation (Quick Sort using a median-of-three)

The choice of the pivot:

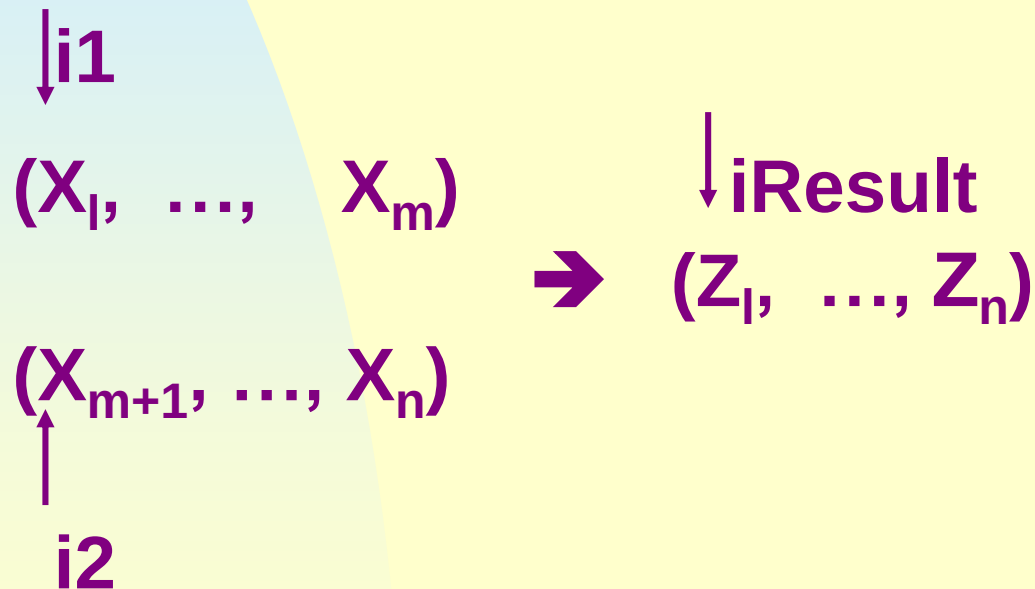
$$\text{pivot} = \text{median} \{R_{\text{left}}, R_{(\text{left}+\text{right})/2}, R_{\text{right}}\}$$

Note that: quick sort is **unstable**.

7.5 Merge Sort

7.5.1 Merging

We firstly discuss how to merge two **sorted lists** to get **a single sorted list**.



Program 7.7: Merging two sorted lists

```
template <class T>
```

```
void Merge (T *initList, T *mergedList,  
            const int l, const int m, const int n)
```

```
{ // two sorted lists initList[l:m] and initList[m+1:n] are  
  // merged to obtain the sorted list mergedList[l:n].
```

```
  for (int i1=l, iResult=l, i2=m+1; i1<=m && i2<=n; iResult++)
```

```
    if (initList[i1] <= initList[i2])
```

```
      mergedList[iResult]=initList[i1++]; //stable
```

```
    else
```

```
      mergedList[iResult]=initList[i2++]; //end of for
```

```
  // copy remaining records , if any, of the first list
```

```
  copy(initList+i1, initList+m+1, mergedList+iResult);
```

```
  // copy remaining records , if any, of the second list
```

```
  copy(initList+i2, initList+n+1, mergedList+iResult);
```

```
}
```

There is record coping, we can eliminate it using link lists.

Analysis of Merge:

- ✓ At each iteration of the for loop, either $i1++$ or $i2++$, the for loop is iterated at most $n-l+1$ times. At most $n-l+1$ records are copied. The total time: $O(n-l+1)$.
- ✓ If each record has a size s , then the total time is $O(s(n-l+1))$.
- ✓ When $s > 1$, use **linked lists**, only need $n-l+1$ links, the merge time becomes $O(n-l+1)$ and is independent of s .

7.5.2 Iterative Merge Sort

Basic idea:

- ✓ At the beginning, interpret the input list comprised of **as n sorted sublists**, each of **size 1**.
- ✓ In the **first merge pass**, these lists are merged by pairs to obtain $n/2$ lists, each of length 2 (if n is odd, then one list is of length 1).
- ✓ In the **second merge pass**, these $n/2$ lists are then merged by pairs to obtain $n/4$ sublists. Each merge pass reduces the number of sublists **by half**.
- ✓ Merge passes are **continued until only one list is get**.

An example:

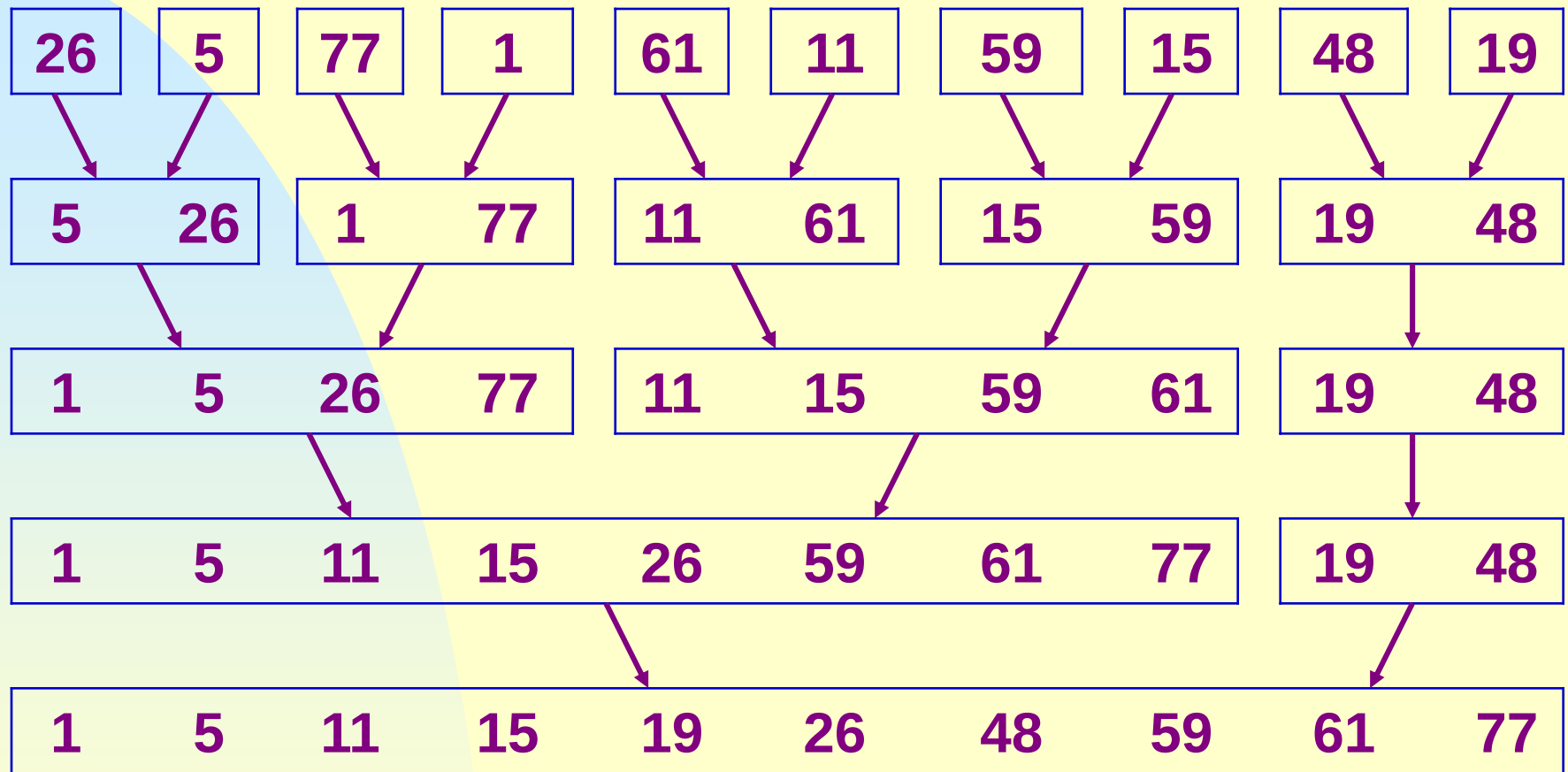
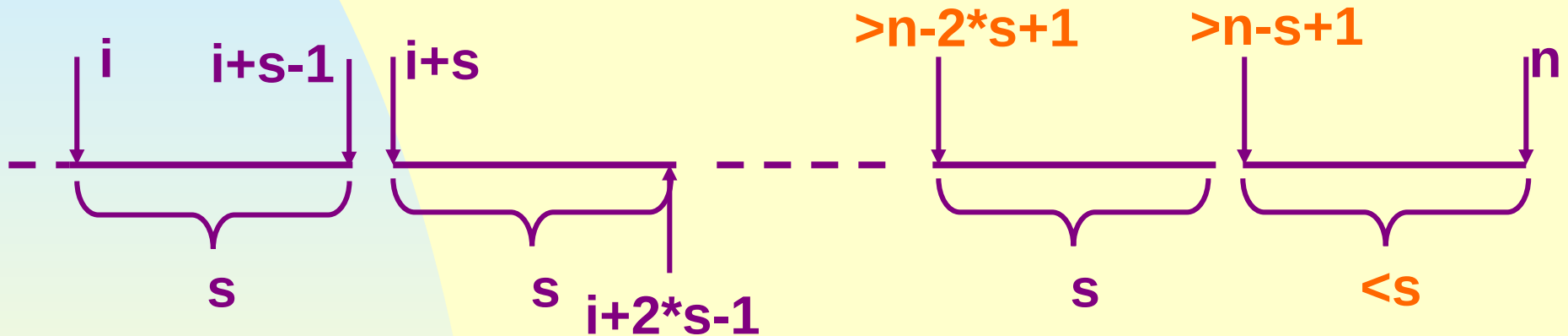


Fig. 7.4 Merge tree

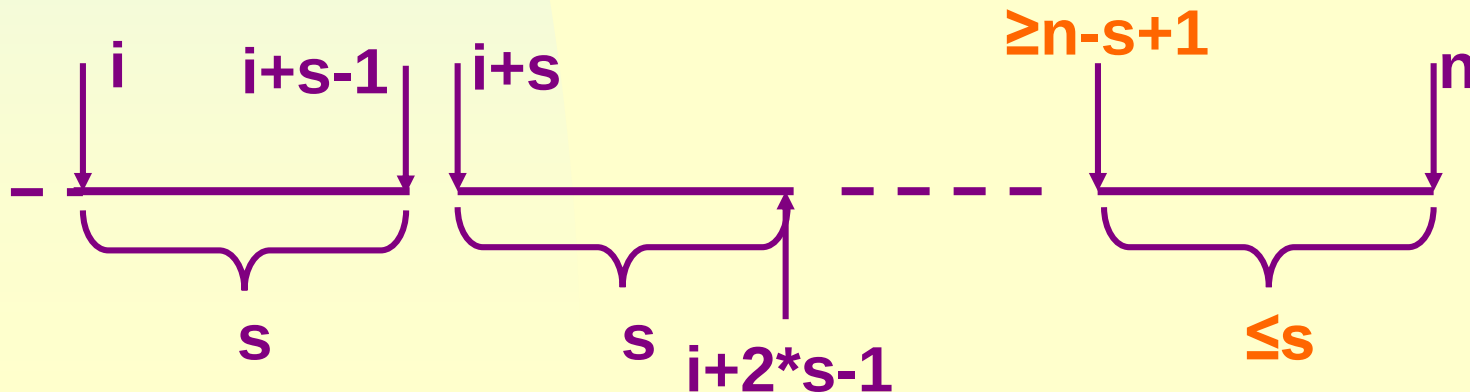
MergePass

Since a merge sort consists of **several merge passes**, it is convenient first to write a **function MergePass** for a merge pass.

- ✓ Case 1: the remaining list of length $>s$ but $<2*s$



- ✓ Case 2: the remaining list of length $\leq s$



```

template <class T>
void MergePass (T *initList, T *resultList,
                 const int n, const int s)
{ // adjacent pairs of sublists of size s are merged from initList
  // to resultList. n is the number of records in initList.
  int i=1 //i is first position in first of the sublists being merged
  for (; i<=n-2*s+1; i+=2*s)//are there 2*s remaining elements?
    Merge(initList, resultList, i, i+s-1, i+2*s-1);

  //case 1: merge the remaining list of length >s but <2*s
  if ((i+s-1)<n) Merge (initList, resultList, i, i+s-1,n);
  //case 2: merge the remaining list of length ≤s
  else copy(initList+i, initList+n+1, resultList+i);
}

```

Note that, even if $n < s$, the algorithm is OK and only copies elements.

MergeSort

Merge sort can be done by repeatedly invoking the merge-pass function **MergePass**.

```
template <class T>
void MergeSort (T *a, const int n)
{ // Sort a[1:n] into nondreasing order.
    T *tempList=new T[n+1];
    //l is the length of the sublist currently being merged
    for (int l=1; l<n; l*=2) {
        MergePass(a, tempList, n, l); //from a to tempList
        l*=2;
        MergePass (tempList, a, n, l); //from tempList to a
        //Note even if n<2l or n<l, MergePass still work.
    } //end of for
    delete [ ] tempList;
}
```

Analysis of MergeSort:

- ✓ Function **MergeSort** makes several passes over the records being sorted:
 - In the 1st pass, lists of size 1 are merged.
 - In the second, the size of the lists being merged is 2.
 - In the i th pass the lists being merged are of size 2^{i-1} .
- ✓ Consequently, a total of $\lceil \log_2 n \rceil$ passes are made over the data. Each pass takes $o(n)$ time, so the total **computing time** is **$O(n \log n)$** .
- ✓ It is easy to verify that MergeSort is **stable**.

7.5.3 Recursive Merge Sort

Basic idea:

- ✓ Divide the list to be sorted into two **roughly equal** parts called **the left and the right** sublists.
- ✓ These sublists are **sorted recursively**.
- ✓ Then the sorted sublists are **merged**.

Example

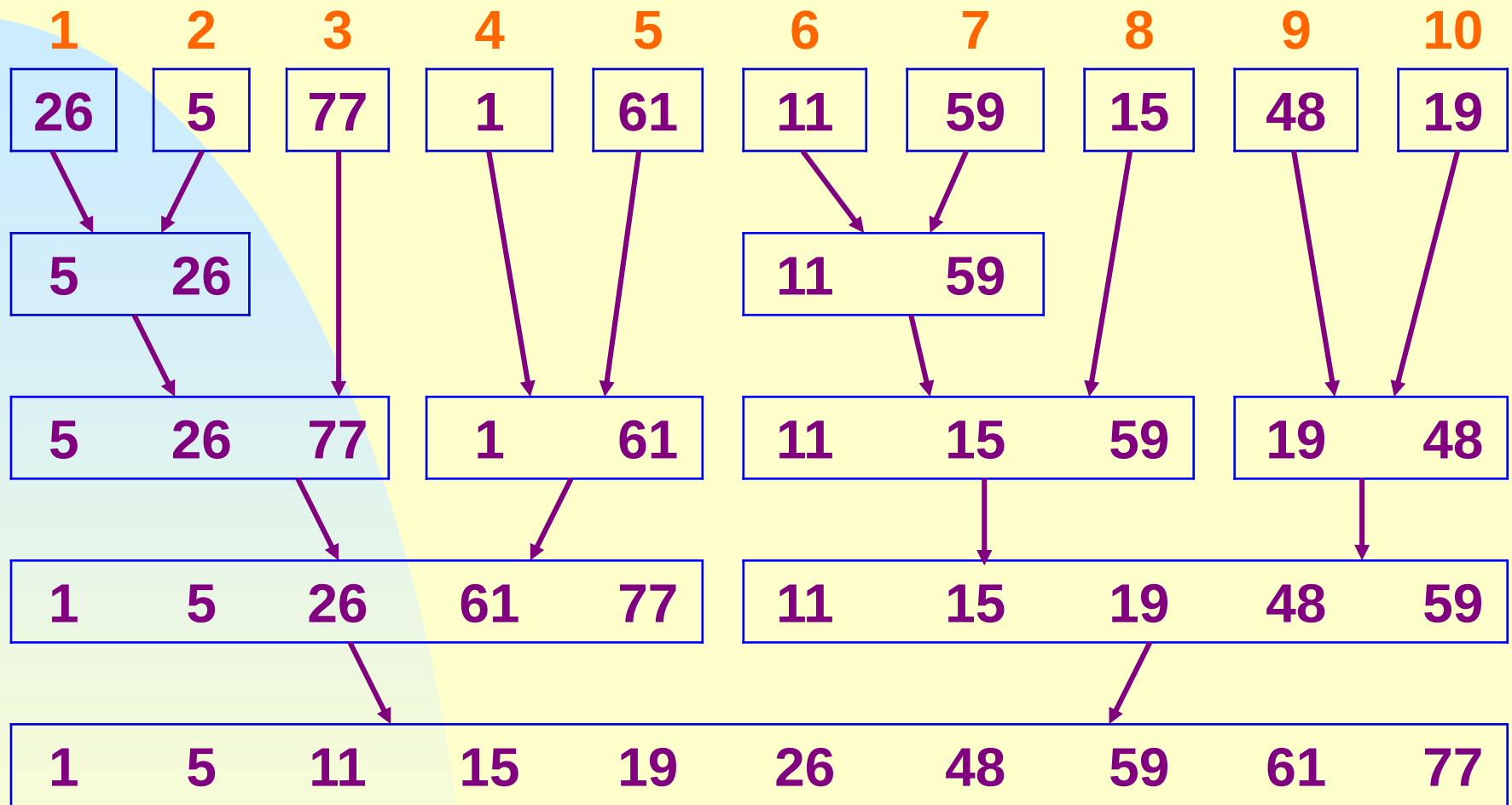


Fig. 7.5 Sublist partitioning for Recursive Merge sort

rMergeSort

Template <class T>

```
int rMergeSort(T* a, int* link, const int left, const int right)
```

```
{//a[left:right] is to be sorted. Link[i] is initially 0 for all i.
```

```
//return the index of the first element in the sorted chain
```

```
    if (left==right) return left;
```

```
    int mid=(left+right)/2;
```

```
    return ListMerge(a, link,
```

```
        rMergeSort(a,link,left,mid),
```

```
        rMergeSort(a,link,mid+1,right) );
```

```
}
```

```
//Here we use ListMerge to merge to eliminate record copy
```

List Merging

- We view the index of each record as an integer pointer to the record.
- Since the records to be merged are treated as sorted sublists, we use an integer array `link[1:n]` where `link[i]` indicates the record following record `i`; `link[i] = 0` if there is no next record.
- Thus, record coping is eliminated and replaced by link changes, and the runtime and additional space required for merging become independent of the size `s` of a record.

Template <class T>

Int ListMerge(T* a, int* link, **const int** start1, **const int** start2)
{//The sorted sublists beginning at start1 and start2 are merged.

//link[0] saves the start of merged chain

int iresult=0; //the last record of the result chain

for(**int** i1=start1,i2=start2;i1&& i2;)

if (a[i1]<=a[i2]) {

link[iResult]=i1;

iResult=i1; i1=link[i1];

}

else{

link[iResult]=i2;

iResult=i2; i2=link[i2];

}

if(i1==0) link[iResult]=i2;

else link[iResult]=i1;

return link[0];

}

	0	1	2	3	4	5
a		10	50	60	20	30



link

	0	1	2	3	4	5
link		5	0	0	3	2

start1=1, start2=4, iresult=0

link

	↓	↓	↓	↓	↓	↓
	0	1	2	3	4	5
link	1	4	3	0	5	2

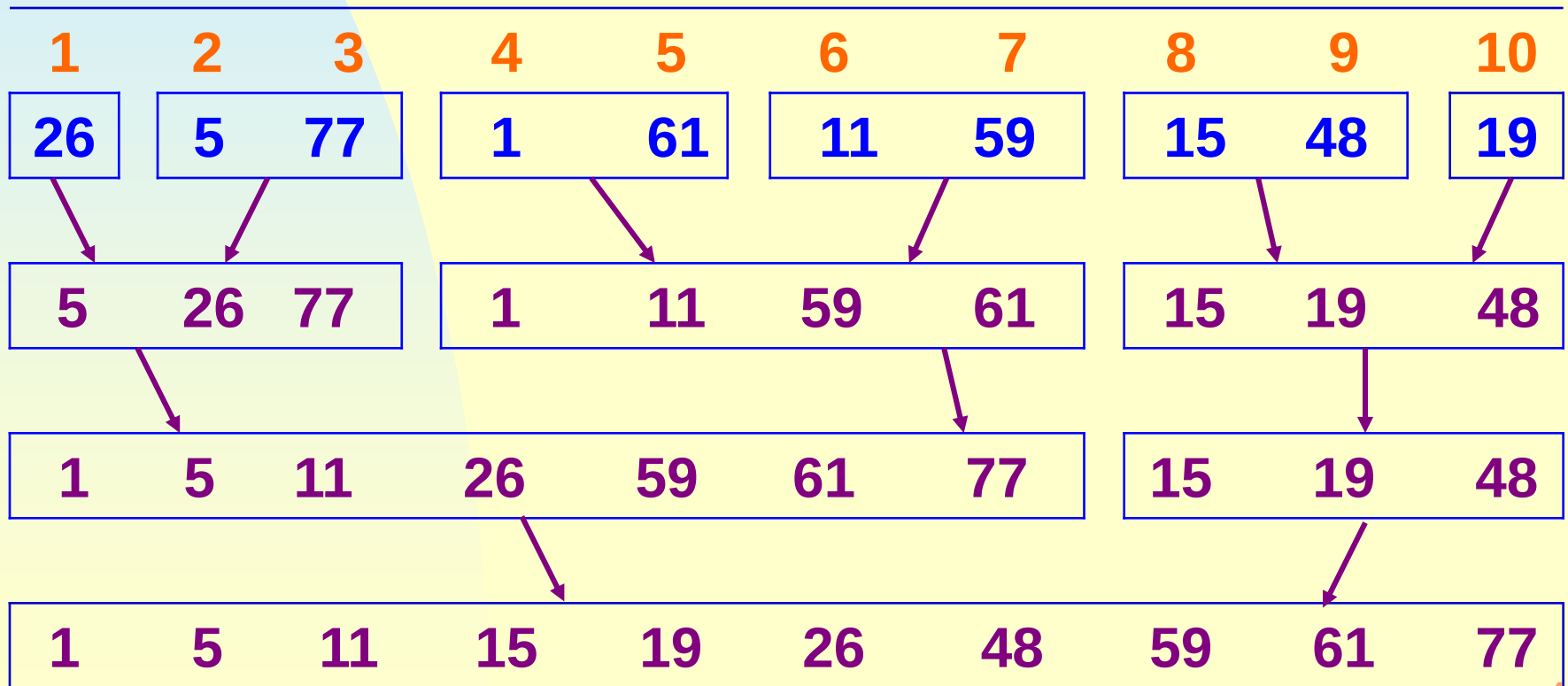
Blue and green stand for two ordered lists respectively.

Analysis of rMergeSort:

- ✓ By using of the ListMerge, its computing time is $O(n \log n)$.
- ✓ The additional space required is $O(n)$.
- ✓ It is easy to see that the recursive Merge Sort is **stable**.

Variation—Natural Merge Sort

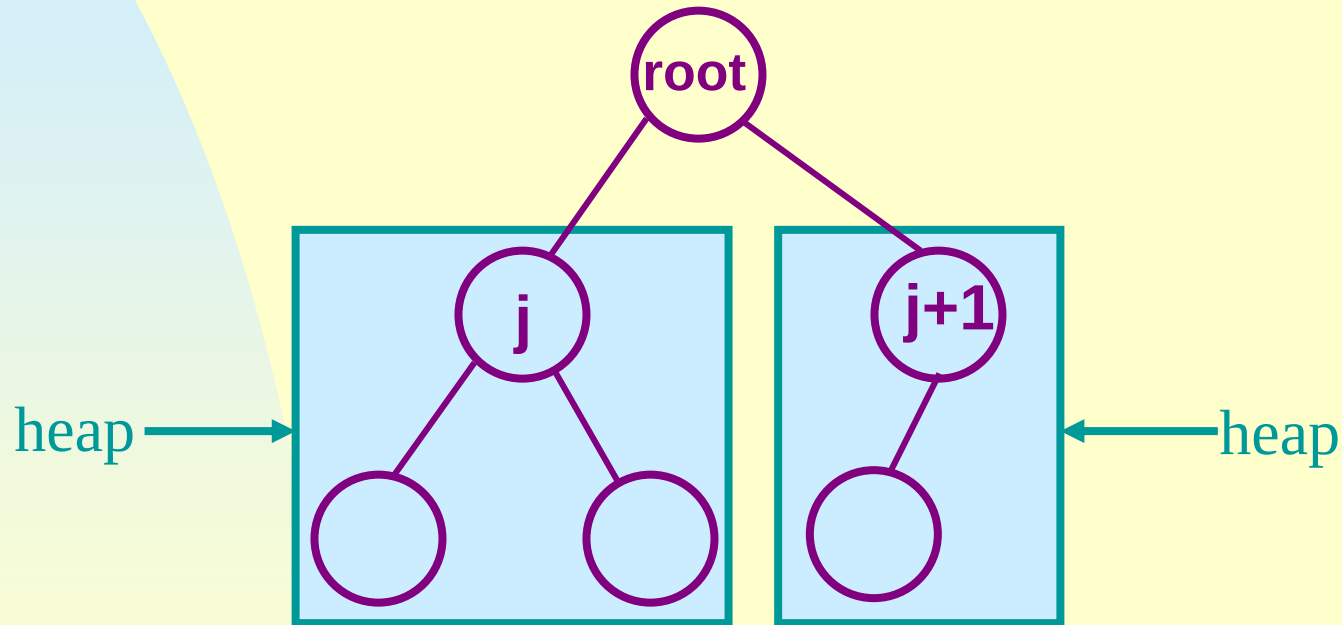
In this implementation, we first make a pass over the data to **identify the already sorted sublists of records**. Merge Sort then utilizes these initial sorted sublists for merging and sorting.



7.6 Heap Sort

- Merge sort has a computing time of $O(n \log n)$, both in worst and average case, but it **requires $O(n)$ additional storage.**
- **Heap sort only requires $O(1)$ additional space,** and also has as its worst and average computing time **$O(n \log n)$.**
- In heap sort, we utilize the **max-heap** structure.
- **We can create a max heap by inserting the records one by one into an initially empty heap.**
- **Question:** Can we create a max heap more efficiently?

- To construct the max-heap **more efficiently**, we utilize the **Adjust** function.
- Adjust rearranges a binary tree whose **left and right subtrees are already max heaps** such that the entire binary tree becomes a max heap.



- By invoking the Adjust function sequentially for nodes from the last non-leaf node up to the root, we can construct a max-heap.

Adjust

```
template <class T>
void Adjust (T *a, const int root, const int n)
{ //Adjust the binary tree with root "root" to satisfy heap property.
//Left and right subtrees of root already satisfy the heap property.
//No node index is > n
    T e=a[root];
    // find proper place for e
    for (int j=2*root; j<=n; j*=2) {
        if (j<n && a[j]<a[j+1]) j++; // let j points to the max child
                                   // if j==n, j has no right sibling

        if (e>=a[j]) break;
        a[j/2]=a[j]; //move jth record up the tree
    } //end of for
    a[j/2]=e;
}
```


Analysis of Adjust:

If the depth of tree is **d**, the **for loop** is executed at most **d times**. Hence, the computing time is **$O(d)$** .

How to sort the list?

(1) Firstly, **create a max heap** by using **Adjust** repeatedly.

(2) Then, execute **n-1** passes: in each pass, **swap the first and last records in the heap**, and then decrease the heap size and **adjust** the heap.

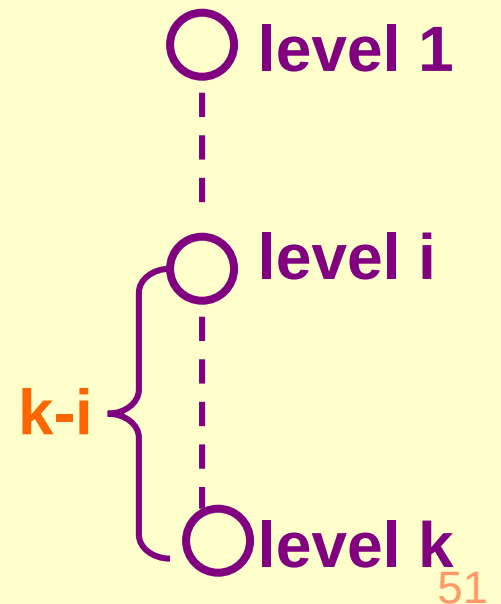
HeapSort

```
template <class T>
void HeapSort (T *a, const int n)
{ // Sort a[1:n] into nondecreasing order.
    for (int i=n/2; i>=1; i--) // Constructing an initial max heap
        // wrong, if it is "for (int i=1; i<=n/2; i++)
        Adjust(a, i, n);
    for (i=n-1; i>=1; i--) // sorting
    {
        swap(a[1], a[i+1]); // swap first and last of current heap
        Adjust(a, 1, i);    //re-heapify
    }
}
```

Analysis of HeapSort:

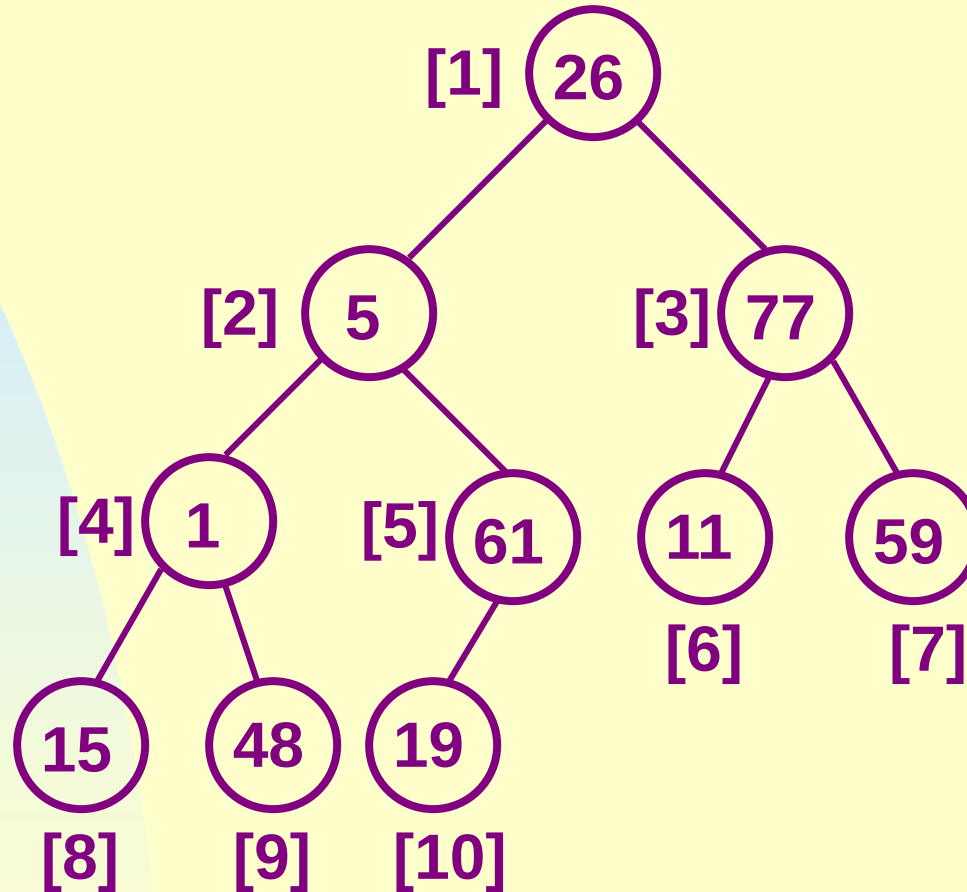
- Suppose $2^{k-1} \leq n < 2^k$, so the tree has **k** levels.
- The number of nodes on level $i \leq 2^{i-1}$.
- In the first “for loop” that constructs an initial max heap, **Adjust** is called once for **each node that has a child**, hence the time of the first “for loop” is **no more than**

$$\begin{aligned} \sum_{1 \leq i \leq k-1} 2^{i-1}(k-i) &= \sum_{1 \leq i \leq k-1} 2^{k-i-1}i \\ &\leq n \sum_{1 \leq i \leq k-1} i / 2^i < 2n = O(n) \end{aligned}$$

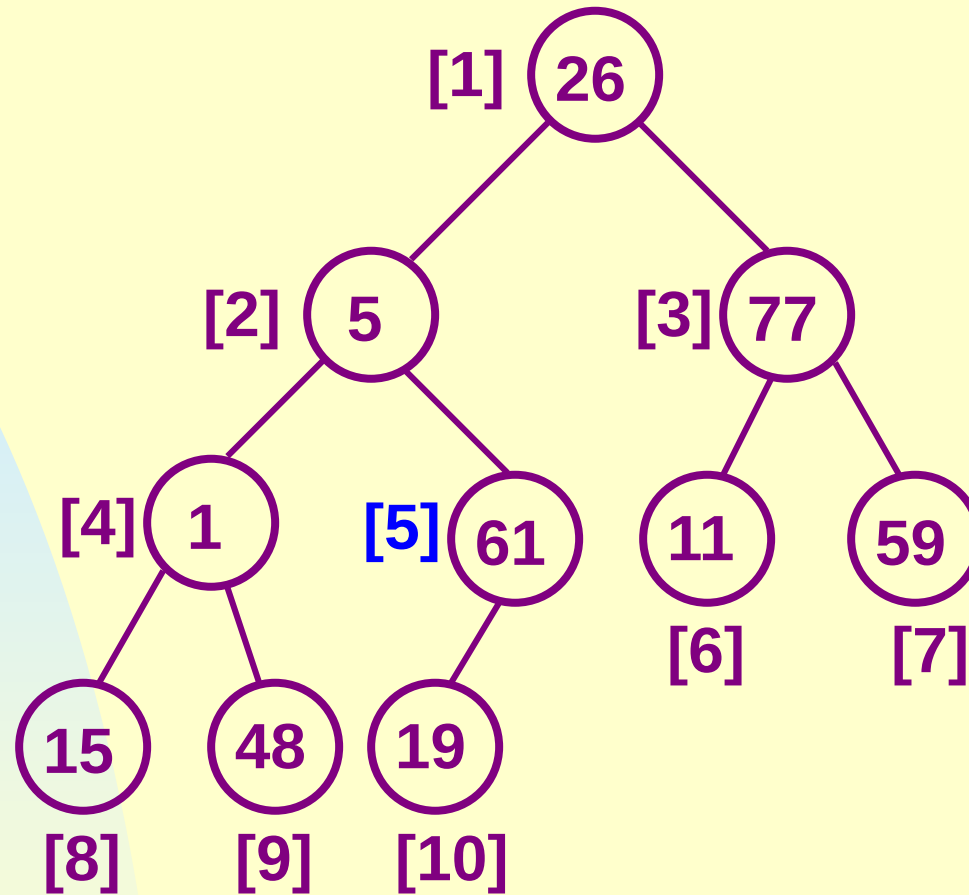


- In the second “for loop”, swap is invoked $n-1$ times, and $n-1$ applications of Adjust are made with each the maximum time $\lceil \log_2(n+1) \rceil$. Hence, the computing time for this loop is $O(n \log n)$.
- So, the total computing time is $O(n \log n)$.
- Additional space: $O(1)$.

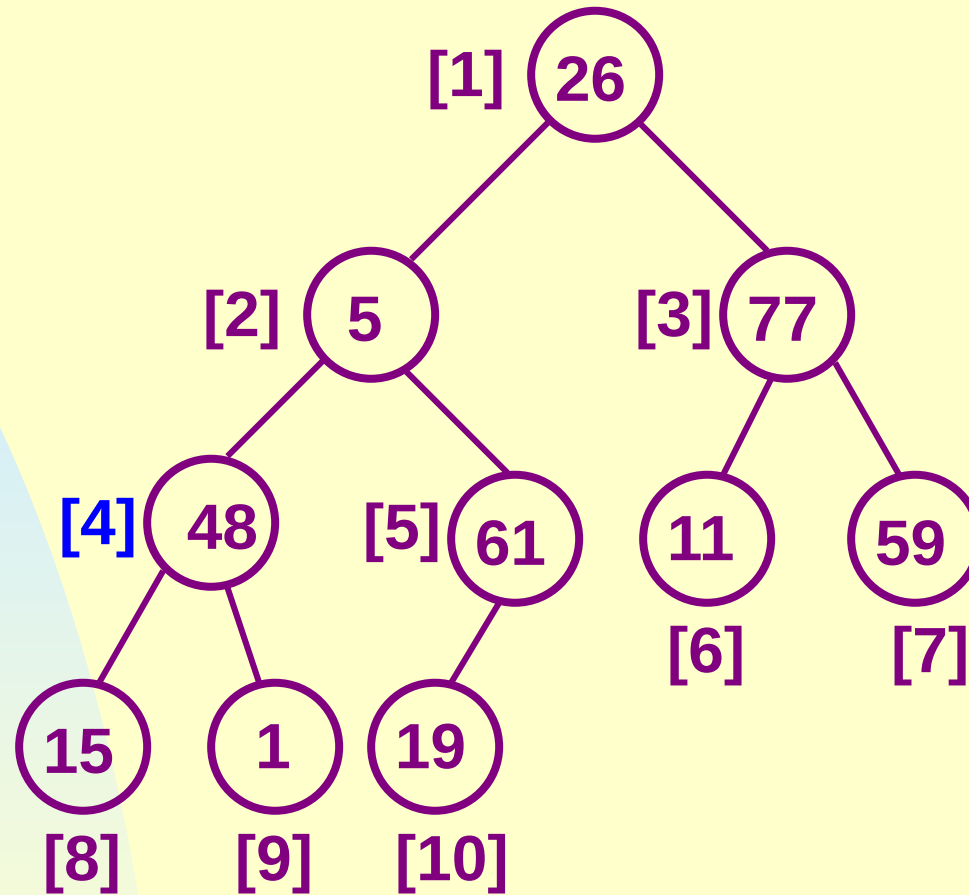
Example 7.7: create a heap and then sort



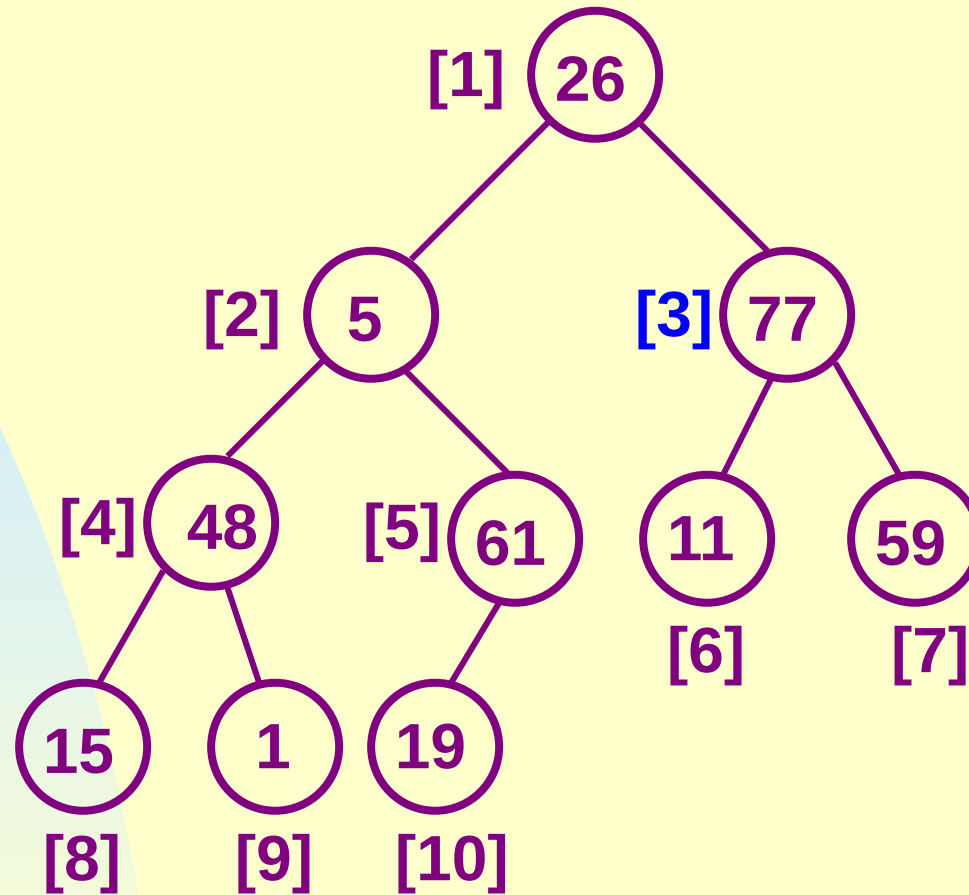
(a) Input array (It is not a heap.)



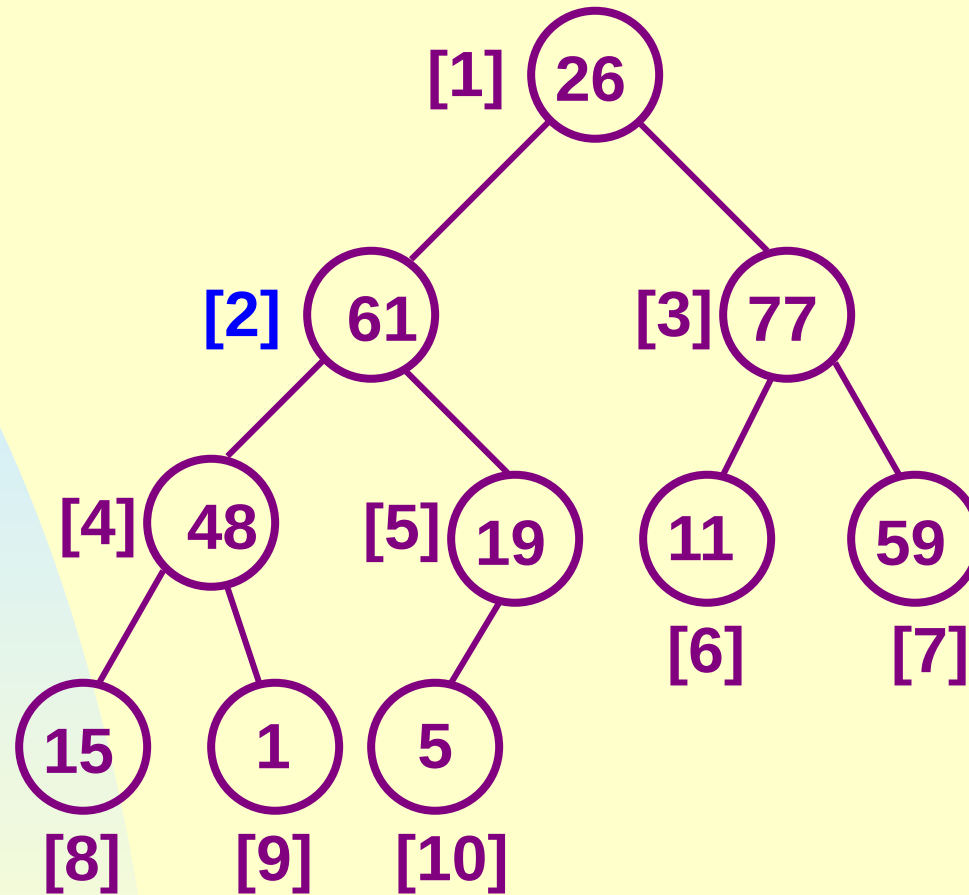
After Adjust (a, 5, n)



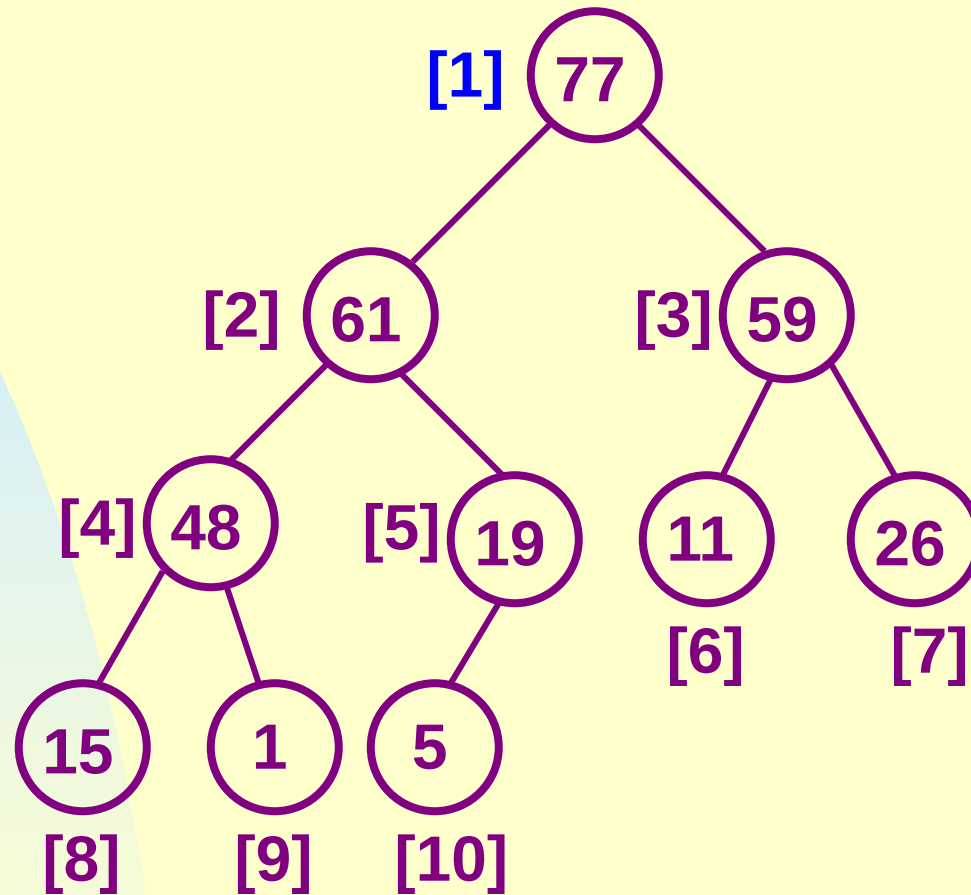
After Adjust (a, 4, n)



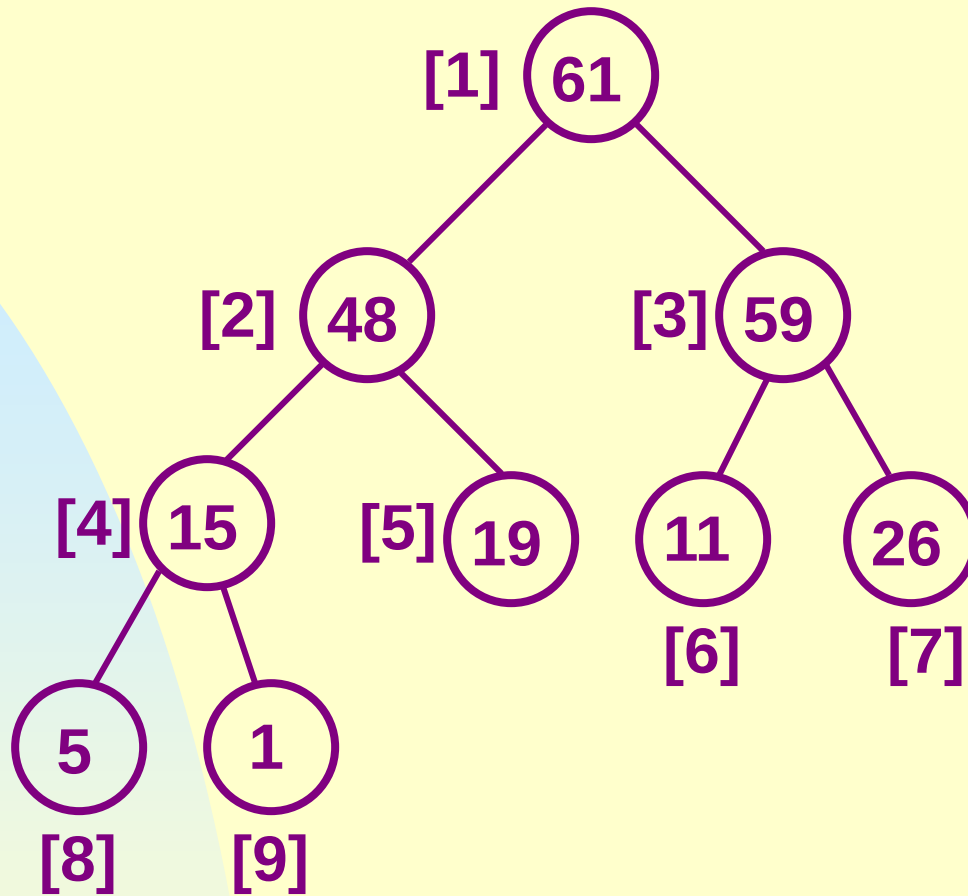
After Adjust (a, 3, n)



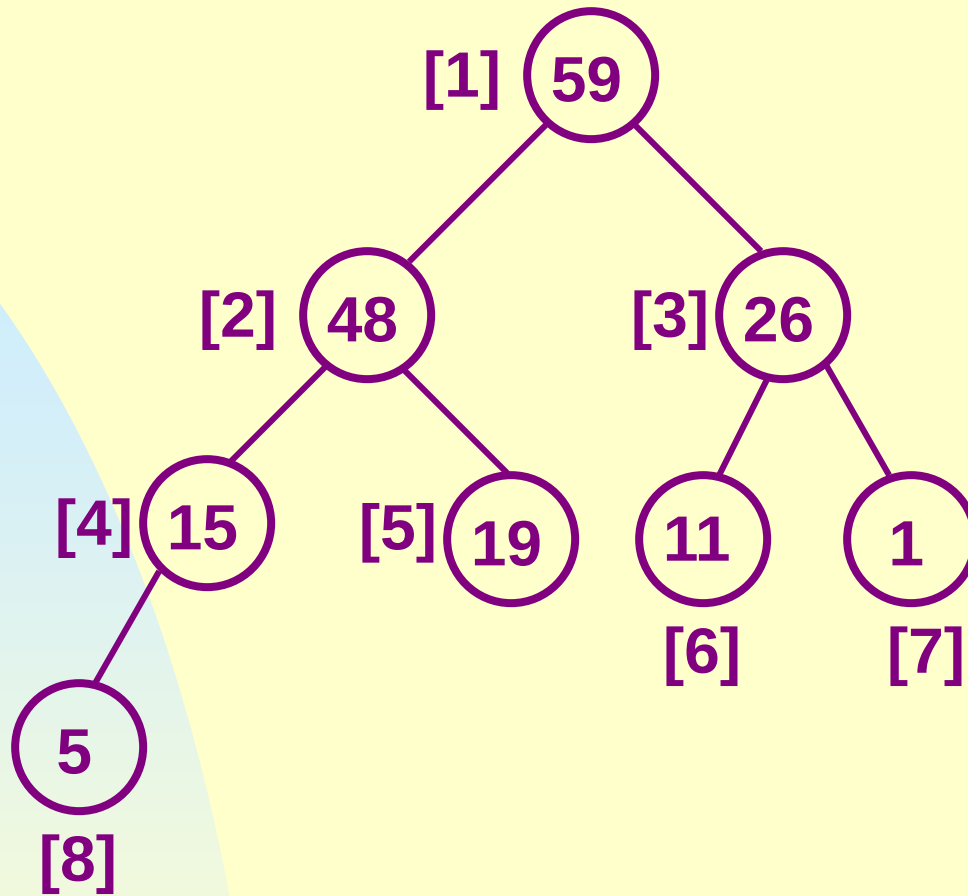
After Adjust (a, 2, n)



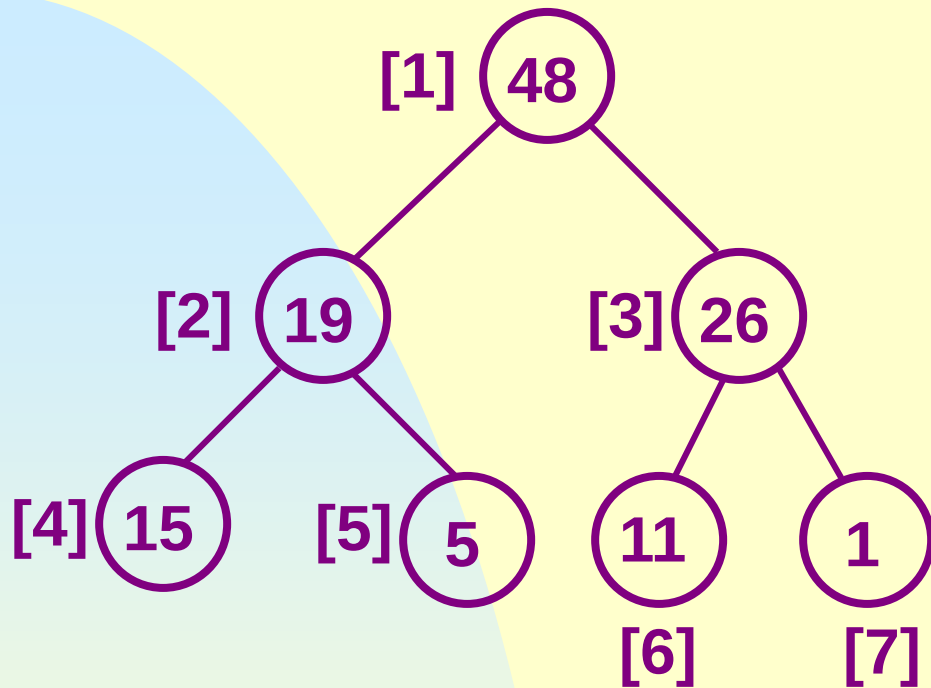
(b) Initial heap (after Adjust (a, **1**, n))



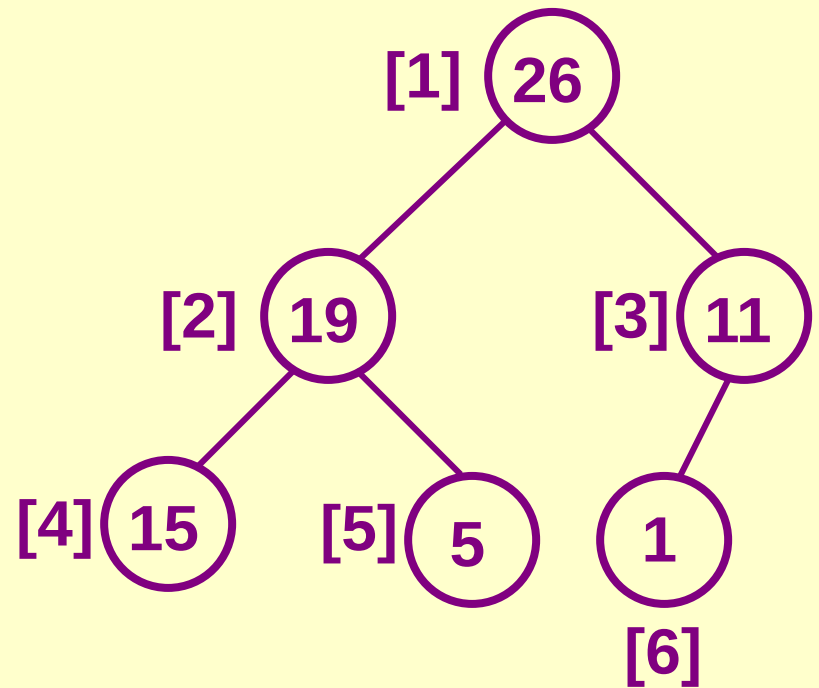
(c) Heap size=9
Sorted=[77]



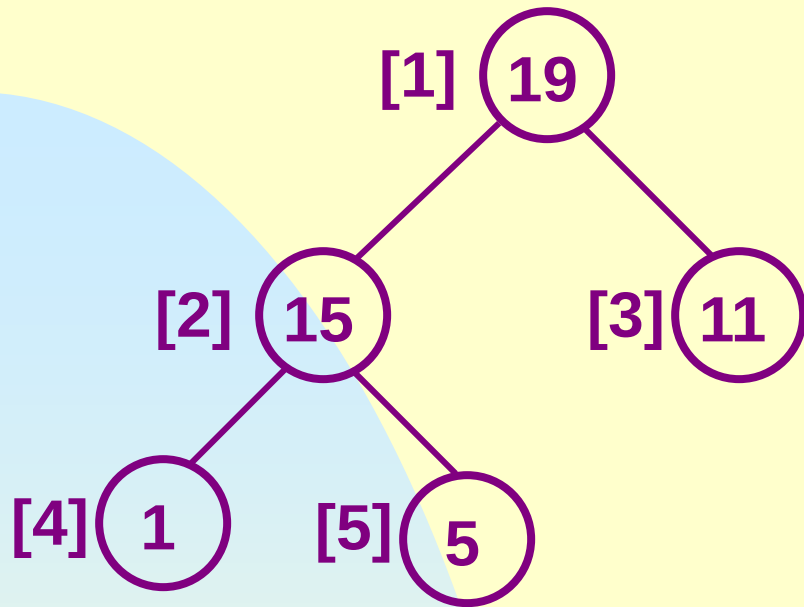
(d) Heap size=8
Sorted=[61, 77]



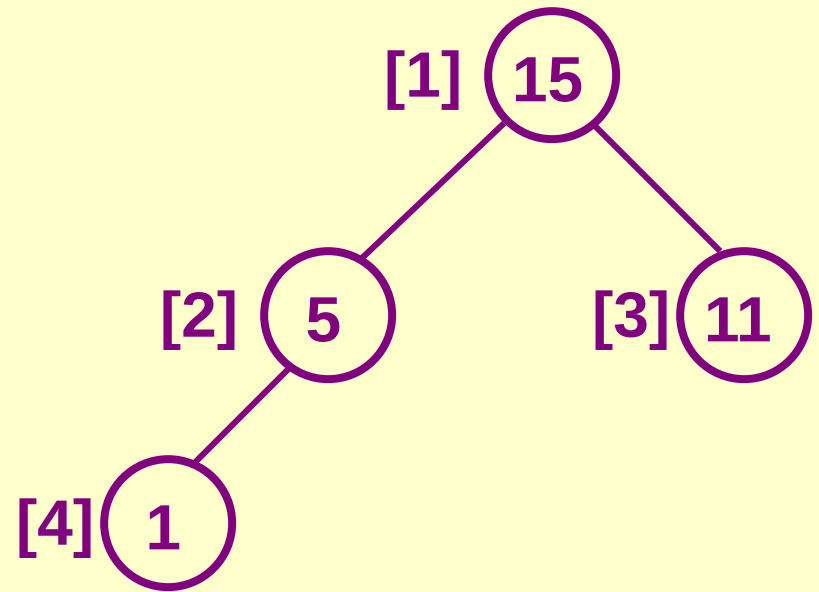
(e) Heap size=7
Sorted=[59, 61, 77]



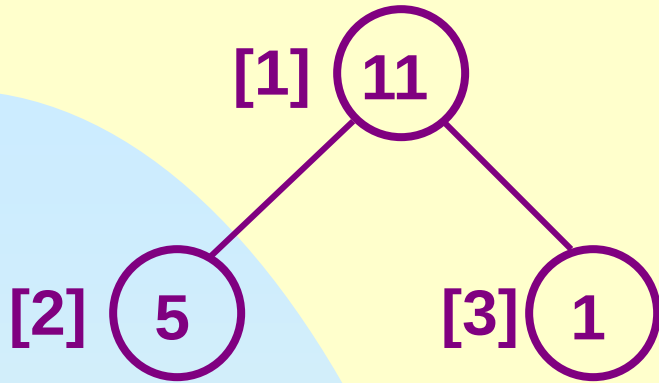
(f) Heap size=6
Sorted=[48, 59, 61, 77]



(g) Heap size=5
[26, 48, 59, 61, 77]

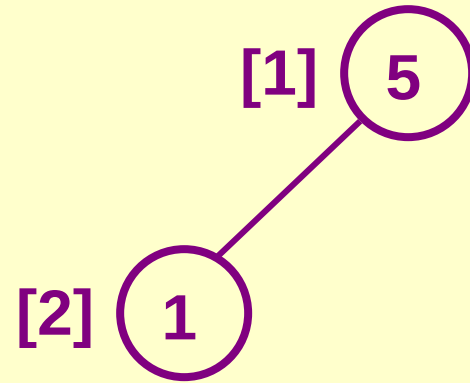


(h) Heap size=4
[19, 26, 48, 59, 61, 77]



(i) Heap size=3

[15, 19, 26, 48, 59, 61, 77]



(j) Heap size=2

[11, 15, 19, 26, 48, 59, 61, 77]



(j) Heap size=1

[5, 11, 15, 19, 26, 48, 59, 61, 77]

7.7 Sorting on Several Keys

- Definition: A list of records $R_1, R_2, \dots, R_n$ is said to be sorted on r keys $K^1, K^2, \dots, K^r$ (K^1 is the most significant key and K^r the least) iff for every pair of records i and j , if $i < j$ then

$$(K_i^1, \dots, K_i^r) \leq (K_j^1, \dots, K_j^r).$$

- The r -tuple $(x_1, \dots, x_r)$ is less than or equal to the r -tuple $(y_1, \dots, y_r)$ iff either $x_i = y_i$ when $1 \leq i \leq j$, and $x_{j+1} < y_{j+1}$ for some $j < r$, or $x_i = y_i$, for $1 \leq i \leq r$.

Example: sorting a deck of cards may be regarded as a **sort on 2 keys**, the **suit and face** values, with the following ordering relations:

K^1 [Suits]:

$\diamond < \clubsuit < \heartsuit < \spadesuit$

K^2 [Face values]:

$2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A$

After sorting, we get the **sorted deck of cards** which has the following ordering:

$\diamond 2, \dots, \diamond A, \clubsuit 2, \dots \clubsuit A, \heartsuit 2 \dots \heartsuit A, \spadesuit 2, \dots, \spadesuit A$

How to sort on multiple keys?

There are **two popular ways** to sort on multiple keys:

- **Most-Significant-Digit-First (MSD)** sort
- **Least-Significant-Digit-First (LSD)** sort

MSD sort idea:

- At first, sort on the **most significant key K^1** to obtain several “**piles**” of records, each having the same value for K^1 .
- Then each of these piles is **independently sorted on the key K^2** to get “**subpiles**”.
- Then each of these subpiles is **independently sorted on the key K^3** to get “**subsubpiles**”.
- ...
- Finally, each pile is **combined**.

LSD sort idea:

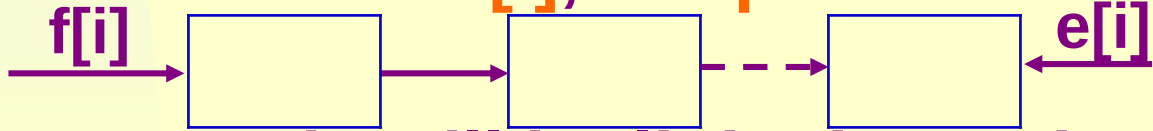
- First, sort the records by the **least significant key K^r** to obtain several piles of records, each having the same value for K^r . Then **collect** the records in each pile sequentially.
- Next, sort the records by the key **K^{r-1}** to obtain several piles of records, each having the same value for K^{r-1} . Then **collect** the records in each pile.
- ...
- Finally sort the records by the key **K^1** to obtain several piles of records, each having the same value for K^1 . Then **collect** the records in each pile.

Remarks on LSD and MSD

- ✓ LSD is simpler to implement than MSD as the piles and subpiles obtained do not have to be sorted independently.
- ✓ LSD or MSD sorting can be used to sort even when the records have only one key.
 - To achieve this, we interpret the key as consisting of several subkeys.
 - For example, if the keys are numeric, then each decimal digit may be regarded as a subkey.
 - e.g., if $0 \leq K \leq 999$, we can use either the LSD or MSD sorts for three keys $(K^1 K^2 K^3)$, $0 \leq K^i < 10$, $i=1,2,3$. The sorting of each subkey can be implemented using a bin sort with 10 bins.

Radix Sort

In radix sort, we assume:

- Records to be sorted are $R_1, R_2, \dots, R_n$.
- Their keys are decomposed using a radix of r , so r bins are needed.
- The original key is decomposed to d digits (d subkeys). Each digit is in the range of $[0, r)$.
- Each record is assigned to the corresponding bin based on its subkey.
- The records in each bin i ($0 \leq i < r$) will be linked together into a chain (queue) with $f[i]$, the pointer to the first record and $e[i]$, the pointer to the last record.


```
graph LR; f[i] --> B1[ ]; B1 --> B2[ ]; B2 -.-> B3[ ]; B3 --> e[i]
```
- Then all the records will be linked together into a chain too.

Radix Sort (using LSD idea)

```
template <class T>
```

```
int RadixSort (T *a, int *link, const int d, const int r, const int n)
```

```
{ // Sort a[1:n] using a d-digit radix-r sort. Each digit is in the  
  // range of [0, r).
```

```
  int e[r], f[r]; // queue end and front pointers
```

```
  // create initial chain of records
```

```
  int first=1, j;
```

```
  for (int i=1; i<n; i++) link[i]=i+1; // linked into a chain  
  link[n]=0;
```

```
  for (int i=d-1; i>=0; i--) //from right to left (LSD)
```

```
  { // sort on digit i
```

```
    fill(f, f+r, 0); // initialize bins to empty queues
```

```
    for (int current=first; current; current=link[current])
```

```
    { // process each record and put it into right queue (bin)
```

```
      int k=digit(a[current], i, r); //digit(e, i, r) returns the ith
```

```
      //radix-r digit from the left (the highest is the 0th digit) of e's key.
```

```
      if (f[k]==0) f[k]=current; // update first pointer of bin k
```

```

        else link[e[k]]=current; //link to the last pointer of bin k
        e[k]=current; //update last pointer of bin k
    } //end of inner for

    //collection the records in each bin
    for (j=0; !f[j]; j++); // find first nonempty queue(bin)
    first=f[j];
    int last=e[j];
    for (int k=j+1; k<r; k++) // concatenate remaining queues
        if (f[k] ) {
            link[last]=f[k]; last=e[k]; //link every nonempty bin
        }
    link[last]=0;
} //end of outer for
return first;
}

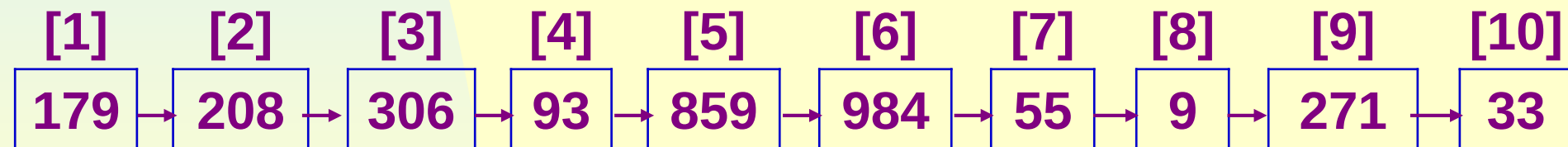
```


Analysis of RadixSort:

- ✓ **d passes** over the data, each taking $O(n+r)$.
Hence the total time is $O(d(n+r))$.
- ✓ d will depend on the choice of the **radix r** and also the **largest key**. Different choices of r will yield different computing times.
- ✓ The total space needed is $O(r)$.

Example 7.8: sort 10 numbers in the range [0,999].

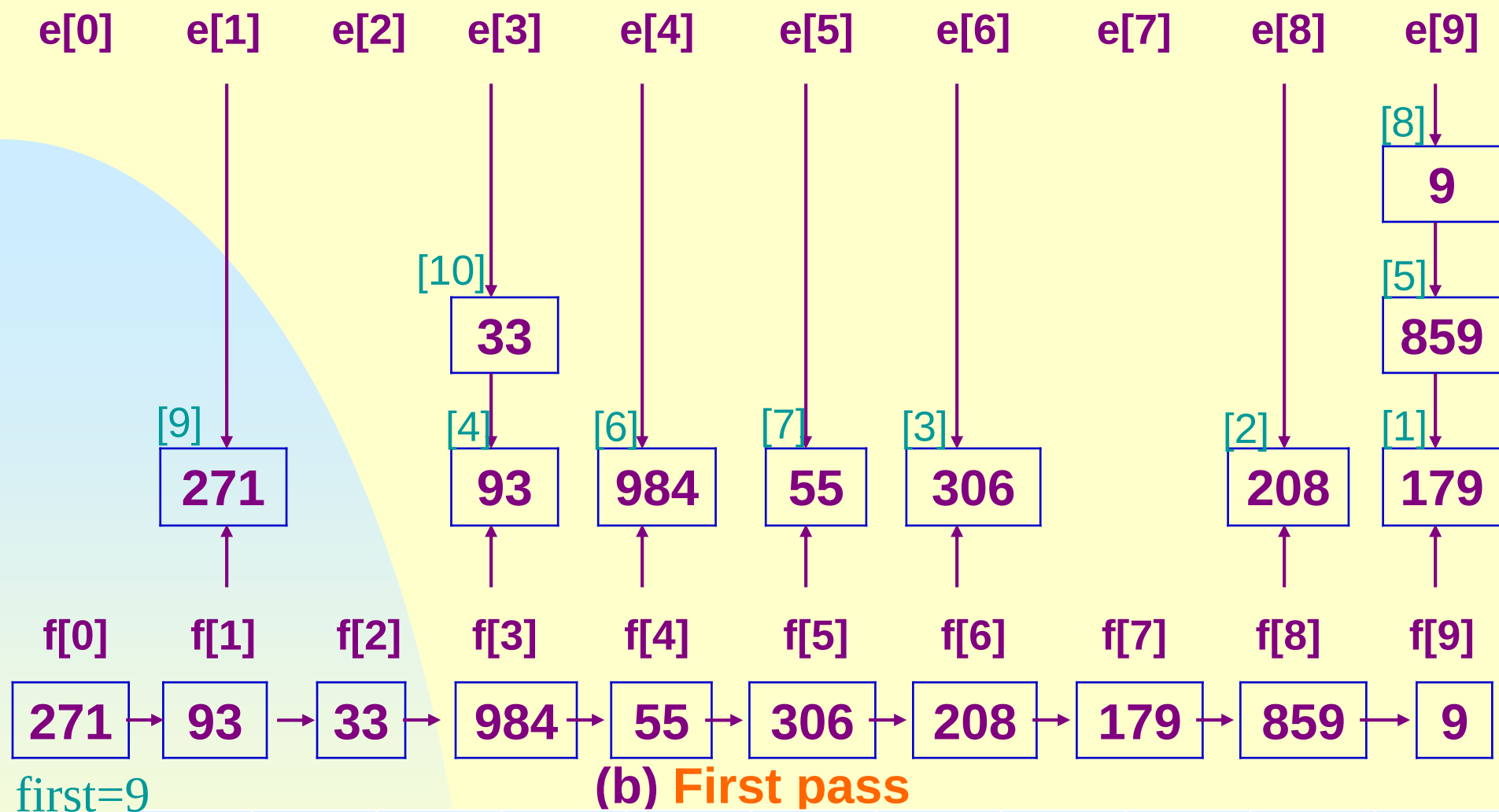
We use $r=10$, hence $d=3$.

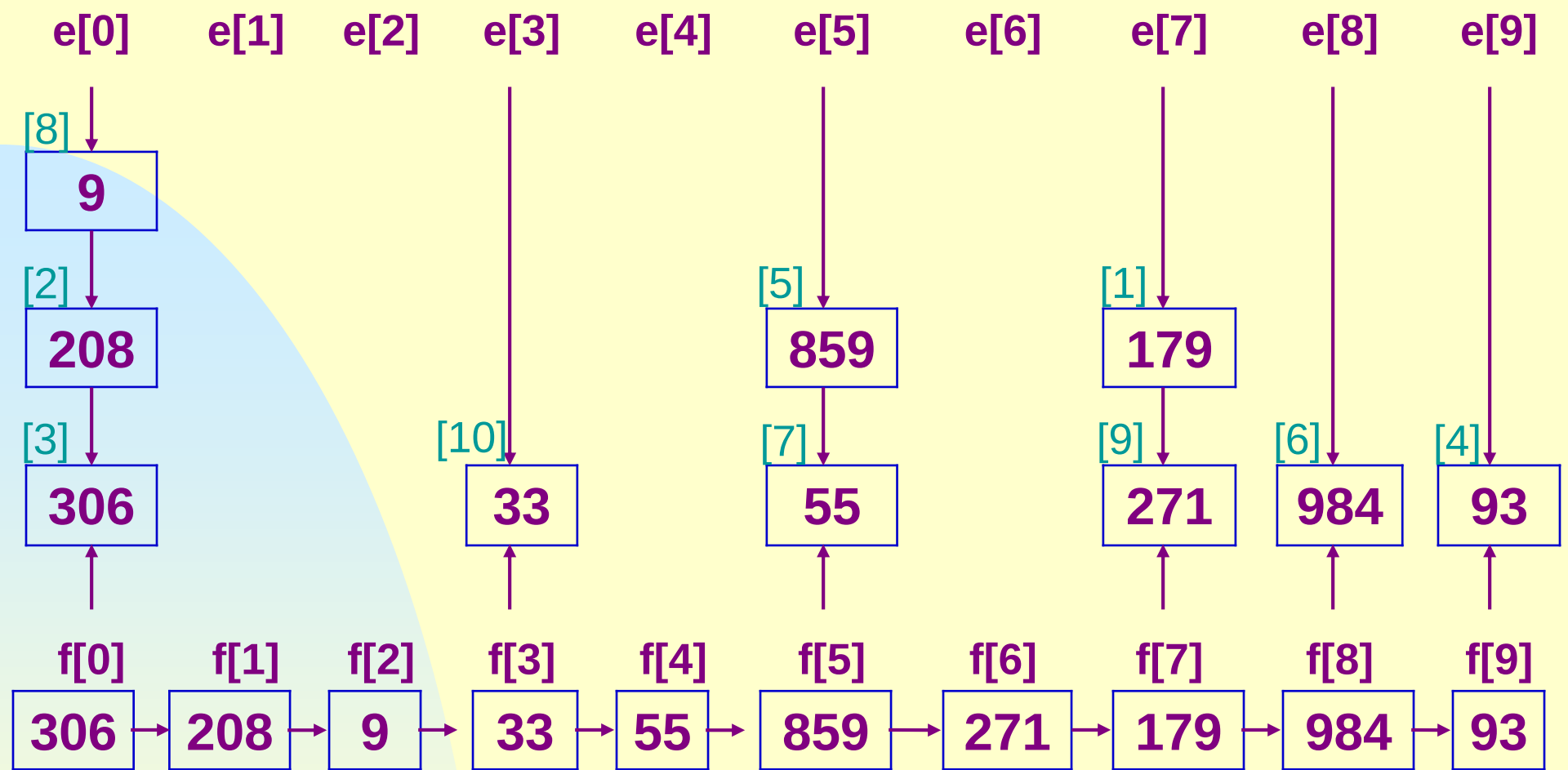


first=1

(a) Initial input

	0	1	2	3	4	5	6	7	8	9	10
a		179	208	306	93	859	984	55	9	271	33
link		2	3	4	5	6	7	8	9	10	0

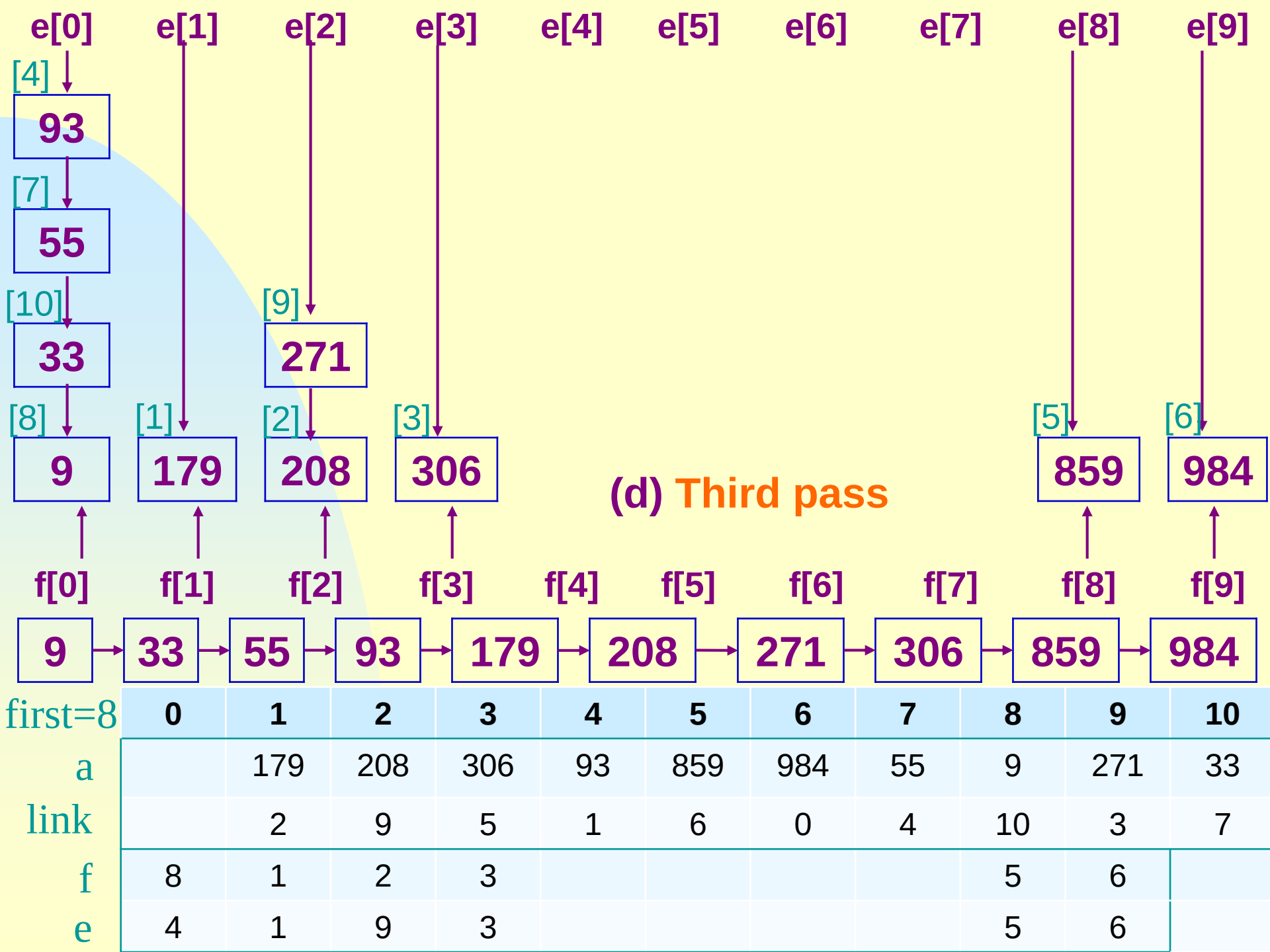




(c) Second pass

first=3

	0	1	2	3	4	5	6	7	8	9	10
a		179	208	306	93	859	984	55	9	271	33
link		6	8	2	0	9	4	5	10	1	7
f	3			10		7		9	6	4	
e	8			10		5		1	6	4	



Radix Sort (using MSD idea)

```
typedef struct {int first; int last;} FL;  
//before sorting link has been initialed to link[i]=i+1 and link[n]=0;  
template <class T>  
FL RadixSort_MSD(int* a, int* link, const int d, const int r,  
                 int first, int last) // Sort a[1:n] using MSD idea.  
{if (d > 0) {  
    int f[r], e[r]; //the first and the last pointers  
    fill(f, f + r, 0); // initialize bins to empty queues  
    //put each record into right queue (bin)  
    for (int current=first; current!=last; current=link[current]) {  
        int k = digit2(a[current], d, r);  
        //returns the d-th digit from right  
        if (!f[k]) f[k] = current;  
        else link[e[k]] = current;  
        e[k] = current;  
    } //end of for
```

```
//sort each nonempty queue(bin) recursively
for (int k = 0; k < r; k++)
    if (!(f[k]==e[k])) { //more than one record in the queue
        fl= RadixSort_MSD(a, link, d - 1, r, f[k], e[k]);
        f[k]=fl.first; e[k] = fl.last; //update f[k] and e[k]
    } //end of if, also end of for
```

```
//collection the records in each bin
```

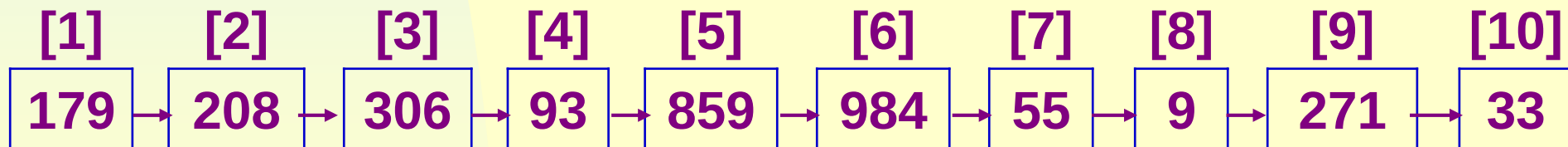
```
int j, l;
for (j = 0; !f[j]; j++); // find first nonempty queue(bin)
first = f[j]; l= e[j];
for (int k = j + 1; k < r; k++) // concatenate remaining queues
    if (e[k]) {
        link[l] = f[k]; l = e[k]; //link every nonempty bin
    } //end of if, also end of for
last = l; link[last] = 0;
} //end of outer if (d > 0)
```

```
return {first, last};
```

Analysis of RadixSort_MSD:

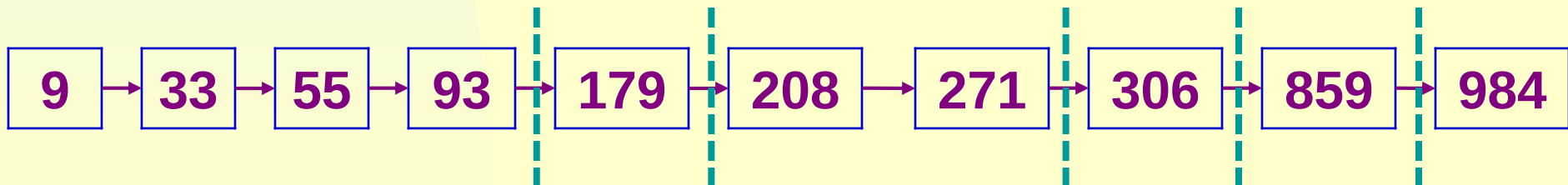
- ✓ Each record is assigned to a bin d times. Since all n records are processed in $O(n + r)$ time per bin-sorting pass, the total time complexity is $O(d(n + r))$.
- ✓ The recursive depth is d , so the total space needed is $O(dr)$.

Example: sort 10 numbers in the range $[0,999]$. We use $r=10$, hence $d=3$.



(a) Initial input

0	1	2	3	4	5	6	7	8	9
93 ₄	179 ₁	208 ₂	306 ₃					859 ₅	984 ₆
55 ₇		271 ₉							
9 ₈									
33 ₁₀									
0	1	2	3	4	5	6	7	8	9
9 ₈			33 ₁₀		55 ₇				93 ₄
0	1	2	3	4	5	6	7	8	9
208 ₂								271 ₉	



(b) final chain

7.9 Summary of Internal Sorting

- ✓ Of the studied sorting methods, **no one method is the best under all circumstances.**
- ✓ **Insertion Sort** is good when the list is already **partially ordered** and also the best sorting method for small n .
- ✓ **Merge Sort** has the **best worst-case behavior** but requires more storage than Heap Sort.
- ✓ **Quick Sort** has the **best average behavior**, but its worst-case behavior is $O(n^2)$.

- ✓ The behavior of **Radix Sort** depends on the **size of keys and the choice of r**.
- ✓ Comparison of **the first four methods** we have studied is as follows:

Method	Worst	Average	Working Storage
Insertion Sort	n^2	n^2	$O(1)$
Heap Sort	$n \log n$	$n \log n$	$O(1)$
Merge Sort	$n \log n$	$n \log n$	$O(n)$
Quick Sort	n^2	$n \log n$	$O(n)$ or $O(\log n)$

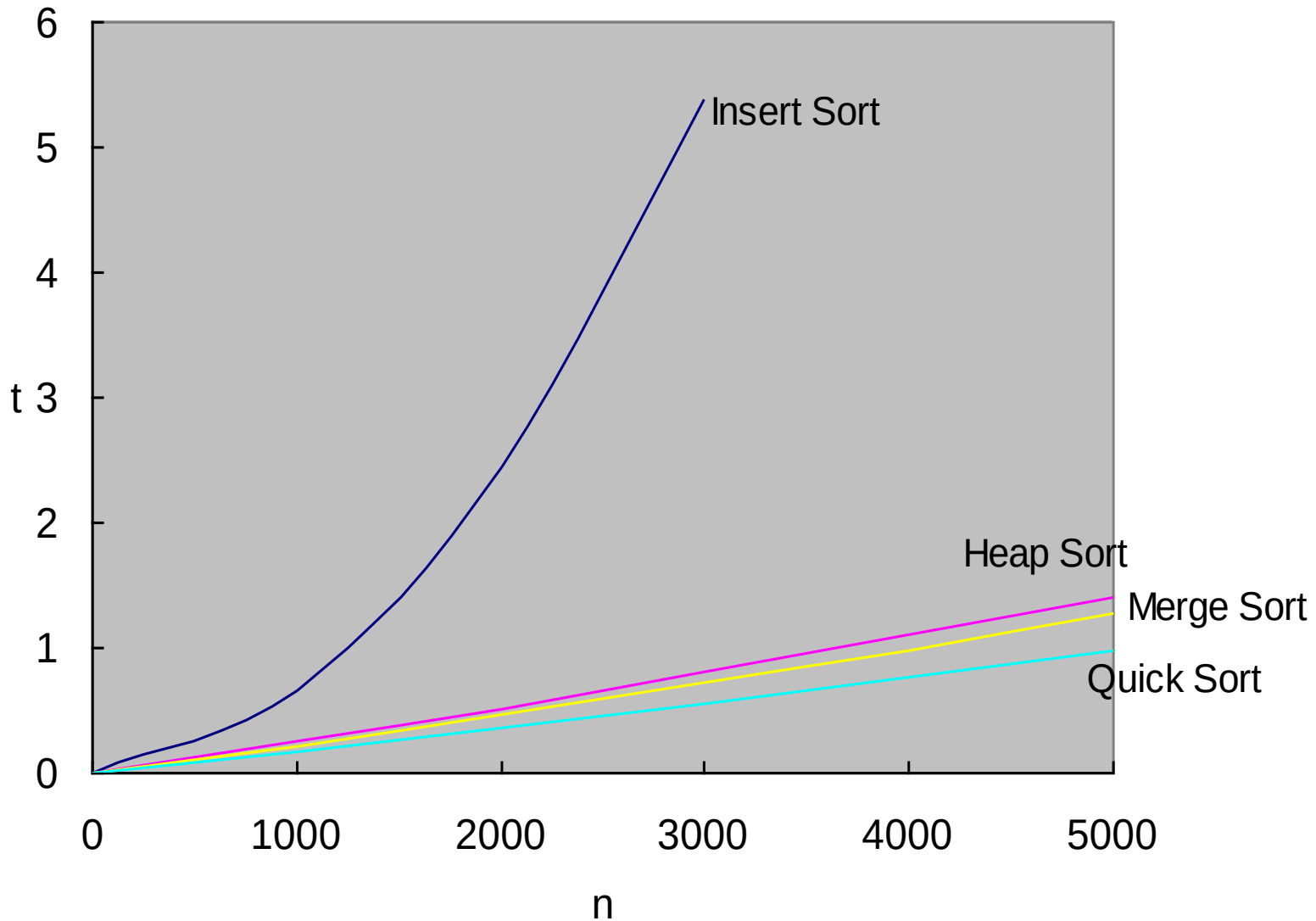
Figure 7.15: Comparison of sort methods

Figure 7.16: Average times for sort methods

n	Insert	Heap	Merge	Quick
0	0.000	0.000	0.000	0.000
50	0.004	0.009	0.008	0.006
100	0.011	0.019	0.017	0.013
200	0.033	0.042	0.037	0.029
300	0.067	0.066	0.057	0.045
400	0.117	0.090	0.079	0.061
500	0.179	0.116	0.100	0.079
1000	0.662	0.245	0.213	0.169
2000	2.439	0.519	0.459	0.358
3000	5.390	0.809	0.721	0.560
4000	9.530	1.105	0.972	0.761
5000	15.935	1.410	1.271	0.970

- Times are in milliseconds
- Times were obtained on a 1.7G Intel Pentium 4 PC with 512M RAM and Microsoft Visual.NET2003.
- For each n at least 100 randomly generated integer instances were run.

**Figure 7.17: Plots of average times
(Times are milliseconds)**



■ As shows above, for average behavior, we can see:

- Quick Sort outperforms the other sort methods for suitably large n .
- The break-even point between Insertion and Quick Sort is between 50 and 100. Let the exact break-even point be n_{Break} .
 - ✓ when $n < n_{\text{Break}}$, Insertion Sort is the best;
 - ✓ when $n > n_{\text{Break}}$, Quick Sort is the best.
- We can improve Quick Sort for $n > n_{\text{Break}}$ by using Insertion Sort to sort the sublists of less than n_{Break} records

■ For **worst-case behavior**, most implementations will show:

- **Merge Sort** to be best for $n > c$ where c is some constant.
- For $n \leq c$ **Insertion Sort** has the **best worst-case behavior**.
- The performance of Merge Sort for $n > c$ can be improved by using Insertion Sort to sort the sublists of less than c records.

7.10 External Sorting

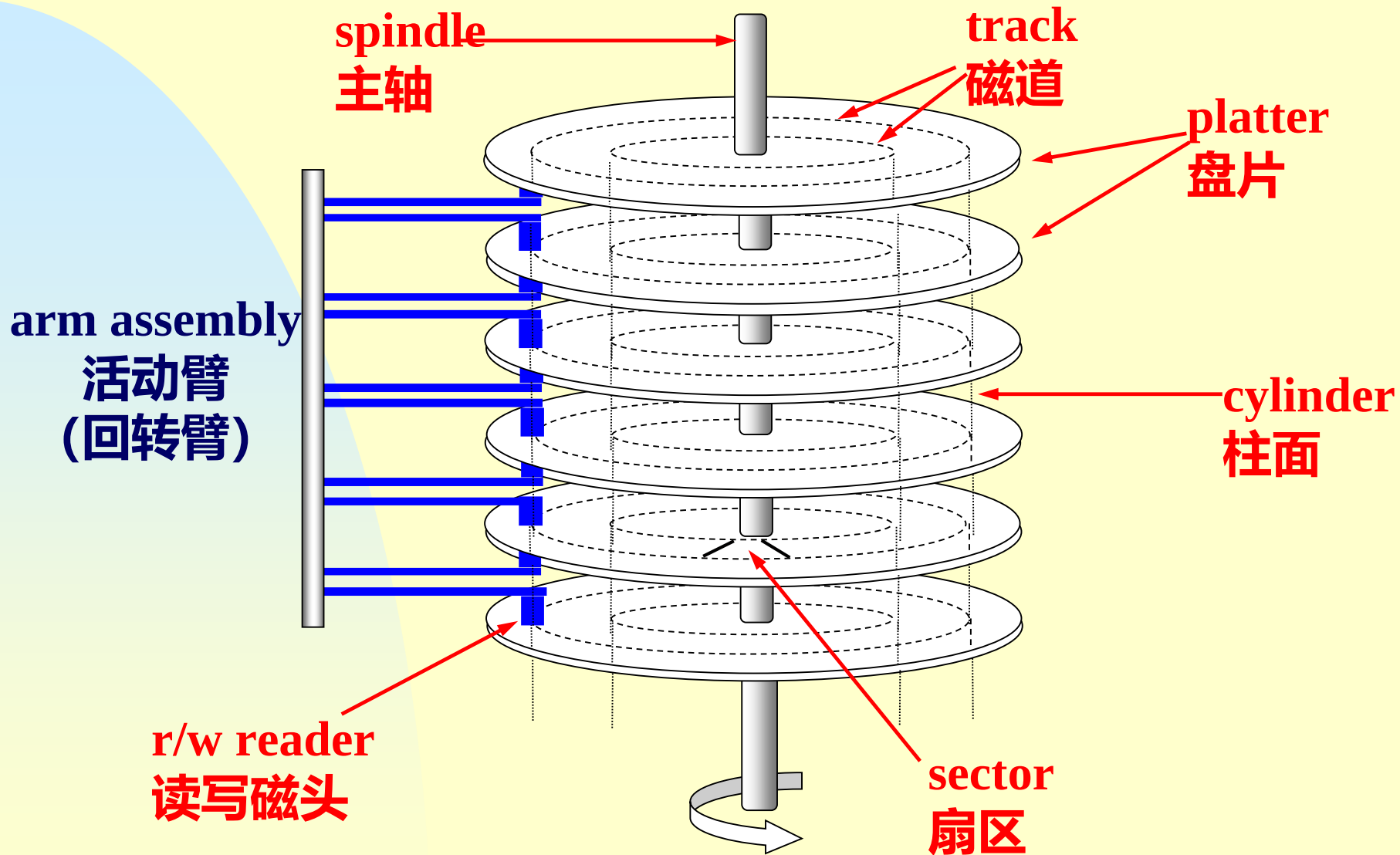
7.10.1 introduction

Assume:

- The lists are **too large** to be entirely contained in the internal memory, making **an internal sort impossible**.
- So, the lists to be sorted resides on a **disk**.
- For disk, **the unit** of data that is read from or written to a disk **at one time is called block**. A block generally consists of several records.

- For a disk, there are **three factors** contributing to the read/write time:
 - (1) **seek time**: time taken to position the read/write heads to the correct **cylinder**
 - (2) **latency time**: time until the right **sector** of the track is under the read/write head.
 - (3) **transmission time**: time to transmit the block of data to/from the disk.

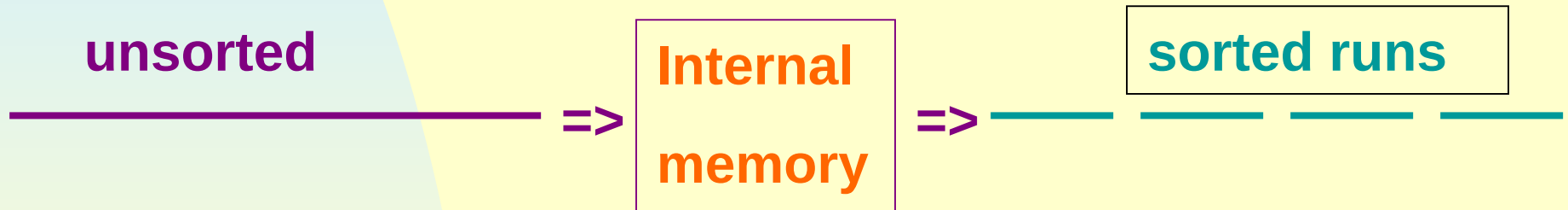
Note that the **read/write rate (I/O rate)** for a disk $\ll$ **CPU rate**.



- The most popular method for **sorting on external storage devices** is **Merge Sort**.
- ✓ Since the function Merge only requires **the leading records of the two runs** being merged to be in memory simultaneously, large runs can be merged efficiently.
- ✓ It is more **difficult** to adapt the **other internal sort methods** considered to external sorting.

■ The external sort consists of **2 distinct phases**:

(1) Segments of the input list are first sorted in memory using an efficient internal **sorting algorithm**, and these sorted segments—referred to as runs—are written to disk as they are generated.



(2) The runs **are merged** together until only one run left.

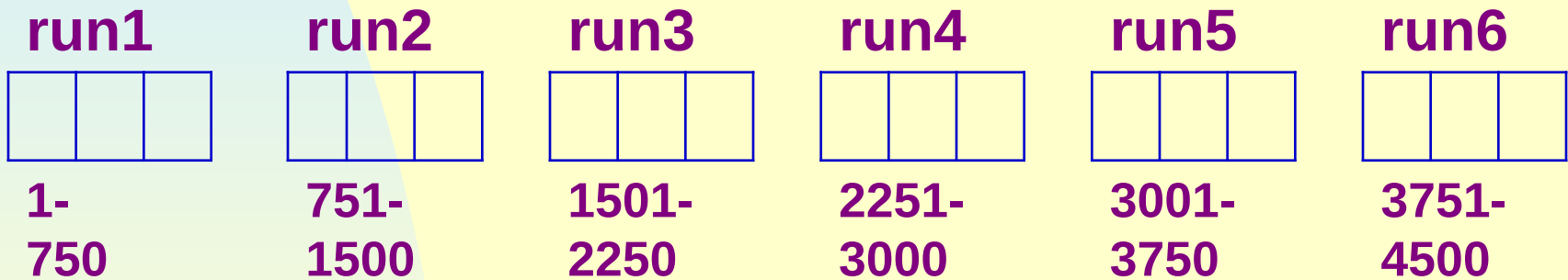
Example 7.12

Assume:

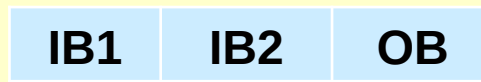
- A list of 4500 records is to be sorted
- The **internal memory** is capable of sorting at most 750 records
- The input list is maintained on disk and has a **block length of 250 records**.
- We have another disk used as a **scratch pad**(暂存器). The input disk is not to be written on.

Now we can **sort the list** as follows:

(1) **Internally sort** 3 blocks (750 records) at a time to obtain **6 runs** R_1 to R_6 . These six runs are written onto the **scratch disk**.



(2) Internal memory is set to three blocks to merge the 6 runs (R_1 and R_2 , R_3 and R_4 , R_5 and R_6). Two blocks of the internal memory will be used as input buffers and the third one is as an output buffer.



Two runs are merged as follows:

- First, read one block of each run into each input buffer;
- Then, the records in the input buffers are merged into the output buffer;
- When the output buffer gets full, it is written onto the disk;
- If an input buffer gets empty, it is refilled with another block from the same run.

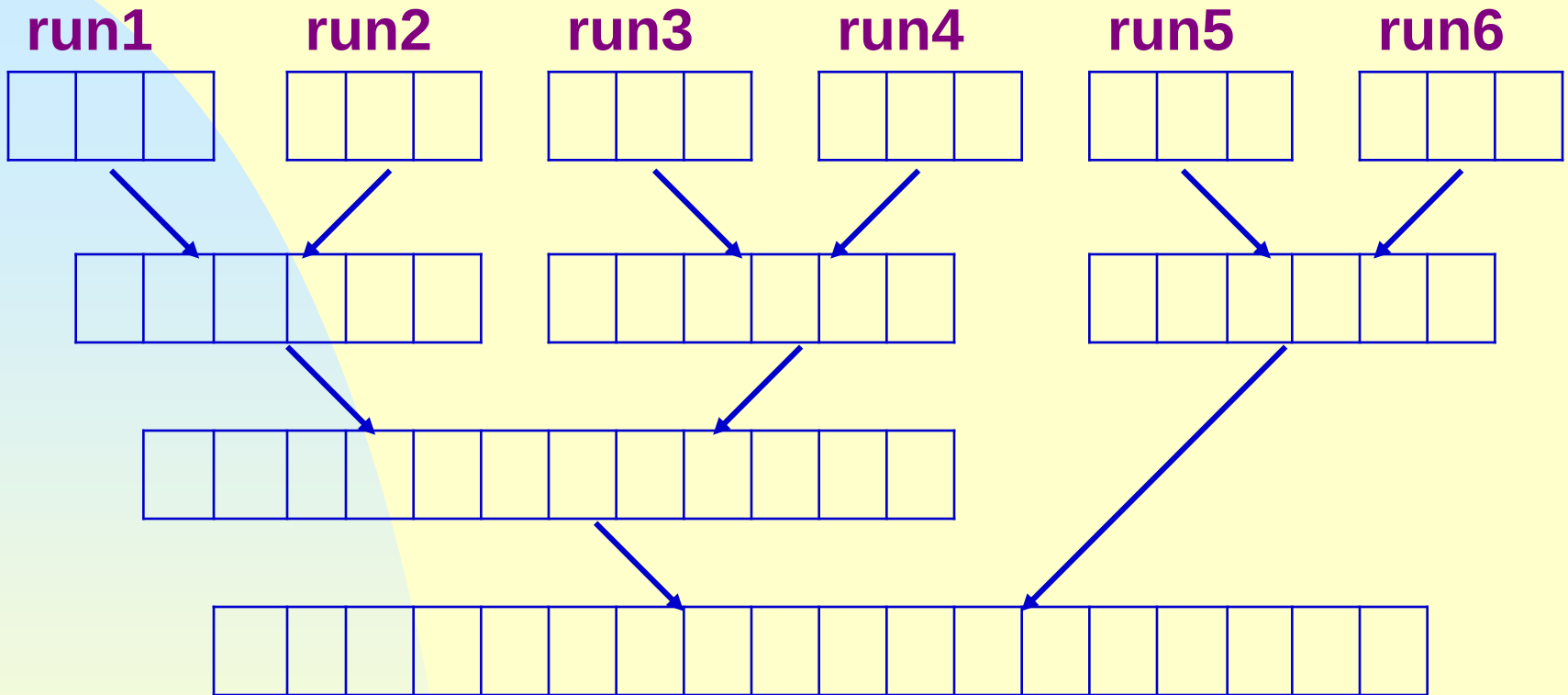


Fig. 7.21 Merging the six runs.

- To analyze the **computing time of external sort**, we use the following notations:

t_s = maximum seek time

t_l = maximum latency time

t_{rw} = time to read or write one block of 250 records

t_{IO} = time to input or output one block

$$= t_s + t_l + t_{rw}$$

t_{IS} = time to internally sort 750 records

$n t_m$ = time to **merge** n records from **input buffers** to **output buffer**

The computing time for the above example:

operation	time
(1) read 18 blocks of input, internally sort, write 18 blocks	$18 t_{IO} + 6t_{IS} + 18 t_{IO}$
(2) merge runs 1 to 6 of 750 records each, 18 blocks	$18 t_{IO} + 4500t_m + 18t_{IO}$
(3) merge 2 runs of 1500 records each, 12 blocks	$12 t_{IO} + 3000t_m + 12 t_{IO}$
(4) merge 1 run of 3000 records with 1 run of 1500	$18 t_{IO} + 4500t_m + 18 t_{IO}$
total time	$132 t_{IO} + 12000t_m + 6t_{IS}$

■ How to **reduce** the computing time?

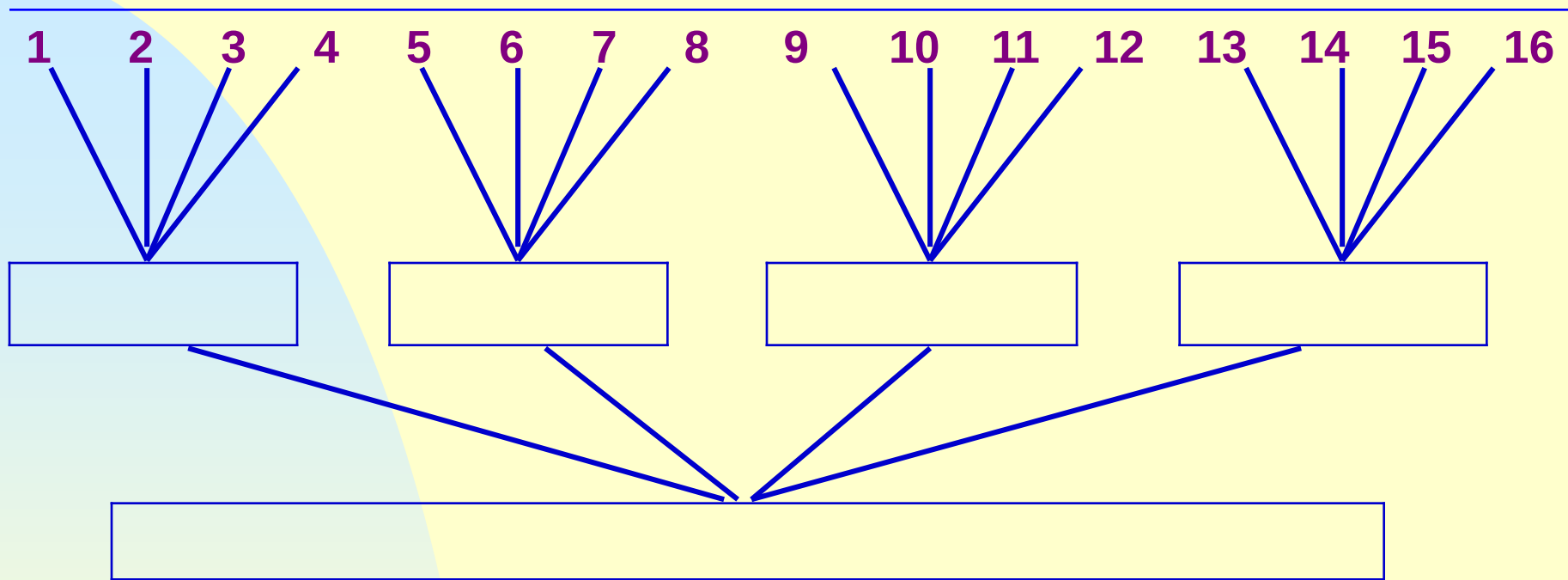
(1) Reducing the number of merge passes over the runs. Because the total I/O time has a close relationship with the number of passes over the records. It can be done by:

- ✓ Using a high-order merge than two-way merge.
- ✓ Generating fewer (or equivalently longer) runs than that generated by the common strategy described above. This can be done using a loser tree.

(2) An appropriate buffer-handling scheme to provide for parallel input, output and merging.

7.10.2 k-way Merging

- ✓ For 2-way merge with **m runs**, a total of **$\lceil \log_2 m \rceil$ passes** (excluding the one for generating the runs) are needed.
- ✓ In general, a **k-way merge on m runs** requires at most **$\lceil \log_k m \rceil$ passes** over the data.
- ✓ As shown in the following merge tree (4-way merge), the number of passes over the data is now reduced to 2:



■ How to do k-way Merging?

- ✓ For large k (say, $k \geq 6$), to select the smallest from k possibilities, we can use a loser tree with k leaves. The time is $\log_2 k$.

■ The total internal k-way merge time:

- ✓ There are n records, the time needed for per level of the merge tree is $O(n \log_2 k)$.
- ✓ There are m runs, the merging tree has $O(\log_k m)$ levels.
- ✓ So, the total internal merge time is

$$O(n \log_2 k \log_k m) = O(n \log_2 m)$$

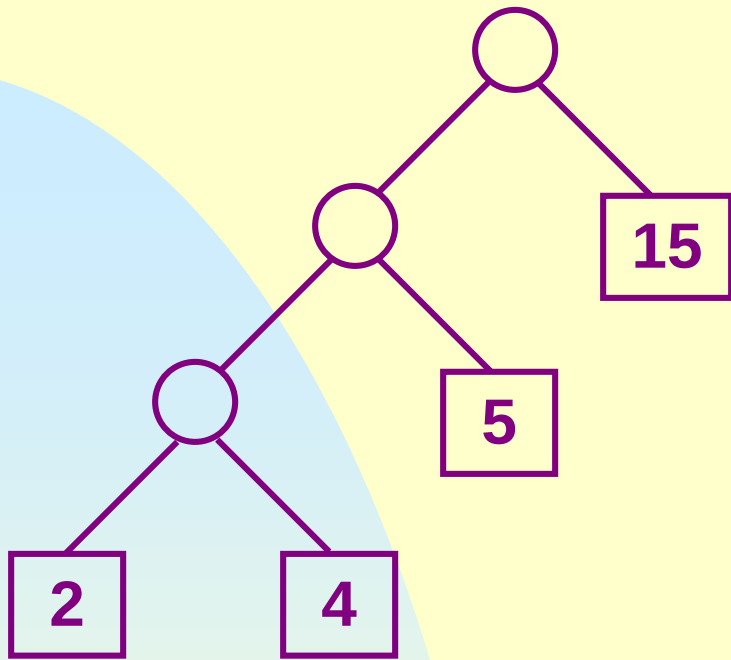
which is independent of k !

■ Comments on high order merge:

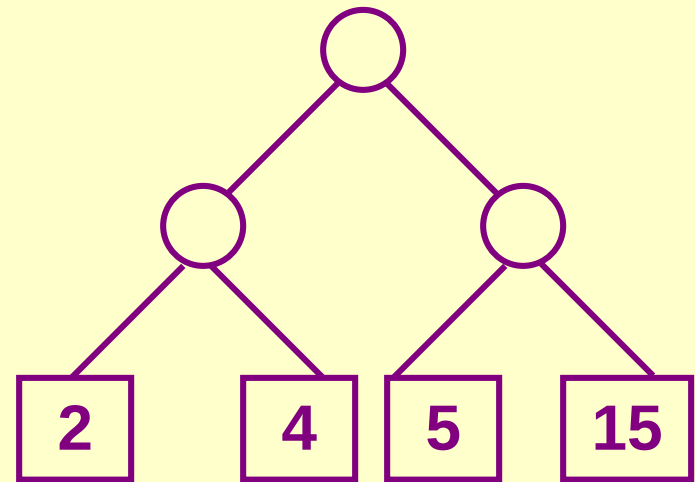
- (1) It **may save** on the amount of I/O being carried out because the number of **passes is reduced**.
- (2) There is no significant loss in internal processing speed.
- (3) But beyond a certain k value, **the I/O time will increase** despite the decrease in the number of passes being made. Because:
 - To do k -way merge we need at least **k input buffers and 1 output buffer** (or the best totally $2k+2$).
 - Since **internal memory** available is **fixed** and independent of k , thus:
 k increases $\Rightarrow$ smaller buffer size $\Rightarrow$ smaller block size $\Rightarrow$ more blocks $\Rightarrow$ increased seek and latency time.

7.10.5 Optimal Merging of Runs

- ✓ In external sort, when runs are of different size, there are **different merging strategies**.
- For example, suppose we have 4 runs of length 2, 4, 5, and 15, respectively. We will show **two ways to merge the 4 runs** using the two-way merging idea:



(a)



(b)

Fig. 7.27: Possible two-way merges

- **internal nodes** --- the circular nodes (represents a two-way merge).
- **external nodes** --- the square nodes (represents the initial runs).

We can see:

- ✓ The number of merges for an individual record is the distance of the corresponding external node to the root.
- ✓ The time required for a merge is linear with respect to the number of records being merged;
- ✓ Thus, the total merge time is proportional to the sum of the products of each run's length and the distance from the root to the corresponding external nodes.
- ✓ This sum is called the weighted external path length E_w .
 - The E_w of (a) is $2*3 + 4*3 + 5*2 + 15*1 = 43$
 - The E_w of (b) is $2*2 + 4*2 + 5*2 + 15*2 = 52$

- **Problem:** How to **minimize the cost (time)** of a k -way merge of n runs with each run having the length q_i , $1 \leq i \leq n$?
- ✓ **Answer:** By building a merge tree of degree k that has **minimum weighted external path length** E_w .
- ✓ We shall consider **$k=2$** only. The result is easily generalized to the case $k > 2$.

- **Another problem:** How to obtain an optimal set of binary codewords for messages $M_1, \dots, M_{n+1}$ such that the expected decoding time is minimized?
- ✓ To understand the question, we first give some detailed descriptions:
 - Each codeword is a binary string that will be used for transmission of the corresponding message.
 - At the receiving end, each codeword will be decoded using a decode tree. A decode tree is a binary tree in which external nodes represent messages.
 - The binary bits in the codeword determine the branching at each level of the decode tree.

---For example, if we interpret '0' as left branch and '1' as a right branch, the decoding tree corresponding to the codewords 000, 001, 01, and 1 (assigned to messages M_1 , M_2 , M_3 , and M_4 respectively) is as follows.

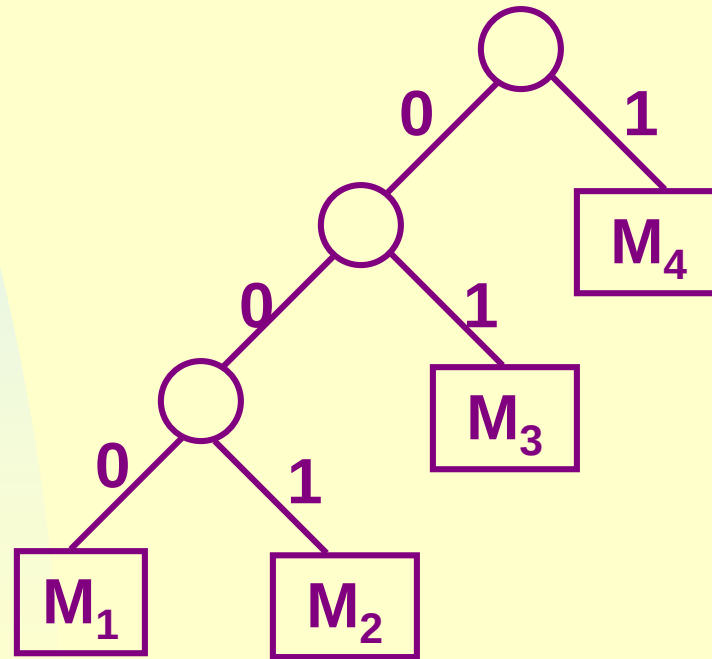


Fig. 7.28 A decode tree

- ✓ The cost of decoding a codeword is proportional to the number of bits in the codeword. This number is equal to **the distance of the corresponding external node to the root node.**
- ✓ Thus, the **expected decoding time** is proportional to

$$\sum_{1 \leq i \leq n+1} q_i d_i$$

where **q_i** is the **relative frequency** for M_i to be transmitted; **d_i** is the **distance** from the root to the external node for M_i .

- ✓ Thus, **the solution** is to select codewords that yield a decoding tree with the minimal weighted external path length.
- ✓ These codes are called **Huffman codes.**

- The above two problems can be summarized as follows: Given a **data set $\{q_i\}$** of weight (probability or frequency), how to build a binary tree with **minimum weighted external path length?**
- ✓ A very nice solution was given by **D. Huffman.**

■ Basic idea of Huffman's solution:

- Start with a **min-heap** consisting of n single-node trees. Each single-node tree corresponds to one of the given data values q_i with its **weight set to q_i** .
- Repeatedly **extract** the two trees with the smallest weights (denoted a and b) from the min-heap; **Combine** them into a single binary tree c by creating a new root node—whose left and right subtrees are a and b (respectively)—and whose weight equals the sum of the weights of a and b . **Insert** the tree c into the min-heap.
- After $n-1$ iterations of this **extract-combine-insert** process, only one binary tree remains in the min-heap. This tree, which has the minimal weighted external path length, is called a **Huffman tree**.

■ Huffman's algorithm

//The function **Huffman** is a friend of **TreeNode** and **MinHeap** which are defined in Chapter 5.

```
template <class T>
```

```
void Huffman(MinHeap<TreeNode<T> *> heap, int n)
```

```
{ //heap is initially a min-heap of n single-node binary trees.
```

```
    for (int i=0; i<n-1; i++) { //loop n-1 times to combine n nodes
```

```
        TreeNode<T> *a=heap.Top();
```

```
        heap.Pop();
```

```
        TreeNode<T> *b=heap.Top();
```

```
        heap.Pop();
```

```
        TreeNode<T> *c=new TreeNode(a→data+b→data, a, b);
```

```
                                //data field of a TreeNode is the weight
```

```
                                //of the binary tree rooted at that node.
```

```
        heap.Push(c);
```

```
    } //end of for
```

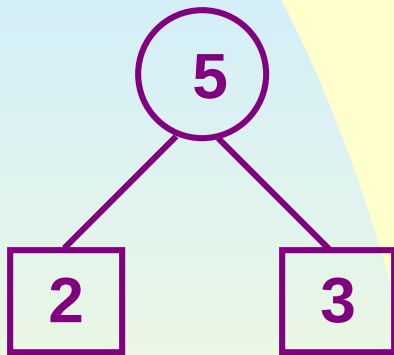
```
}
```


Analysis of Huffman:

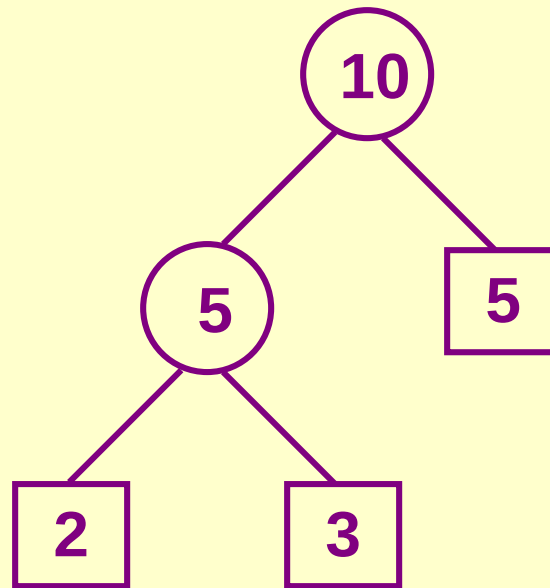
- The main loop --- $n-1$ times
- Each call to Top is $O(1)$, to Pop and Push is $O(\log n)$
- The total time: $O(n \log n)$.

Example 7.15: Construction of a Huffman tree

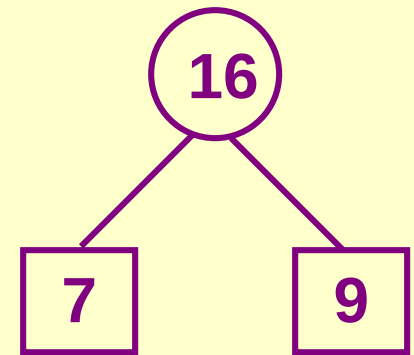
Suppose we have the **weights** $q_1=2$, $q_2=3$, $q_3=5$, $q_4=7$, $q_5=9$, and $q_6=13$, then we can get the sequence of binary trees as follows:



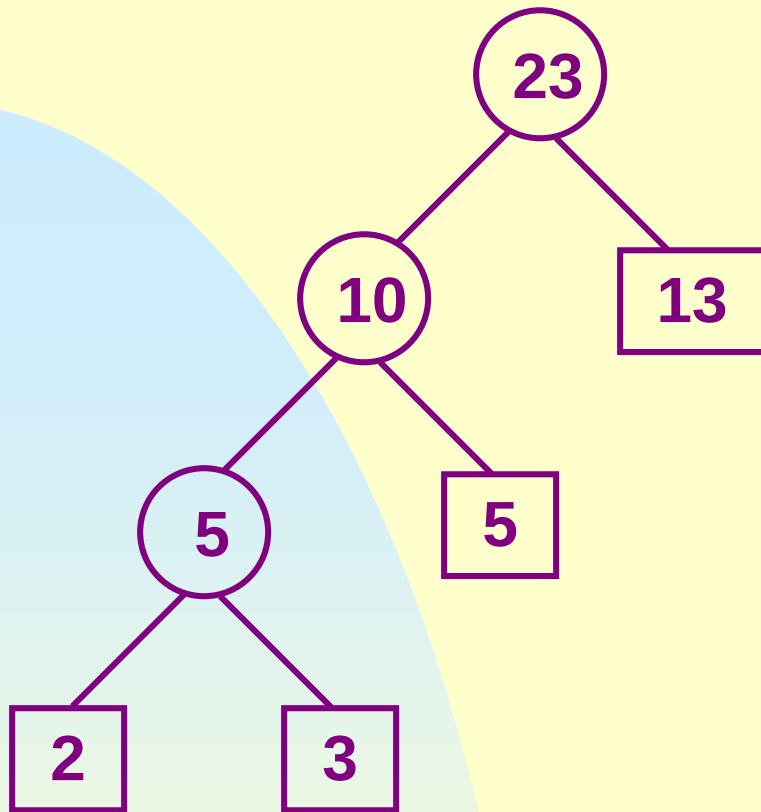
(a) q_1+q_2



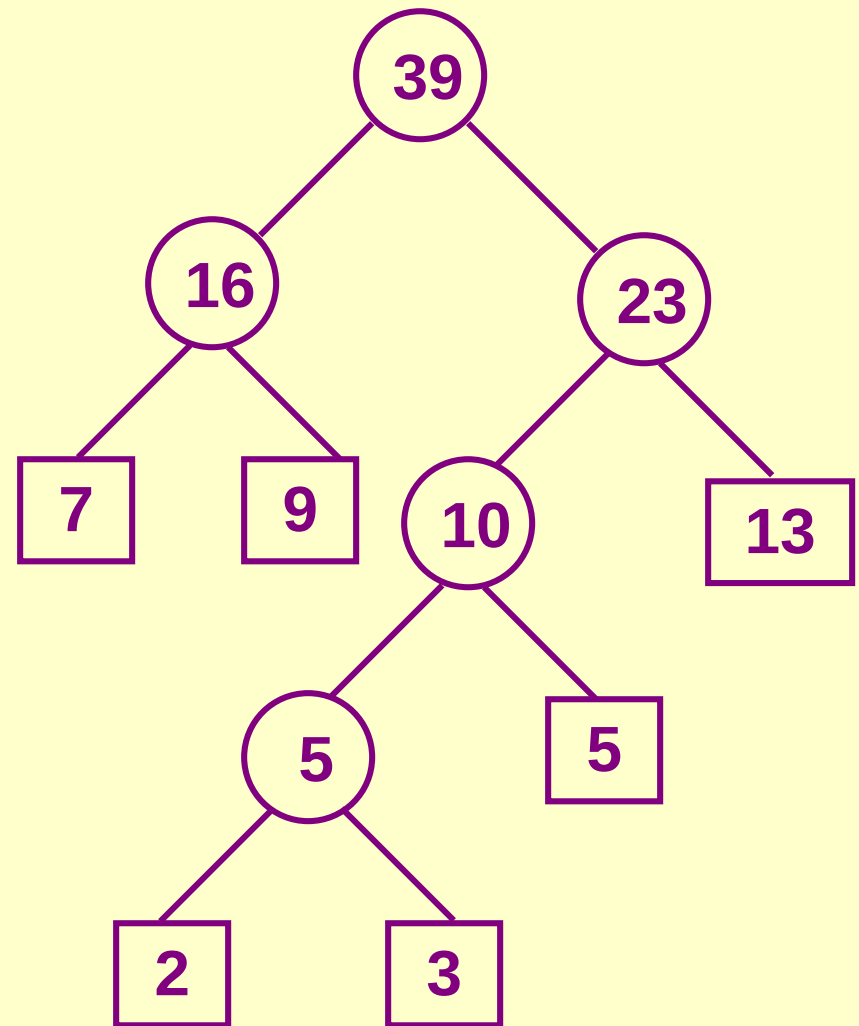
(b) (a)+ q_3



(c) q_4+q_5



(d) (b)+ q_6



(e) (c)+(d)

The weighted external path length of this tree is
 $2 \cdot 4 + 3 \cdot 4 + 5 \cdot 3 + 13 \cdot 2 + 7 \cdot 2 + 9 \cdot 2 = 93$

Exercise(s) and Experiment(s)

Exercise(s):

- **Write the C++ functions for (a) Insertion Sort; (b) Quick Sort; (c) Merge Sort; (d) Heap Sort; (e) Radix-r Sort**

Experiment(s):

- **none**